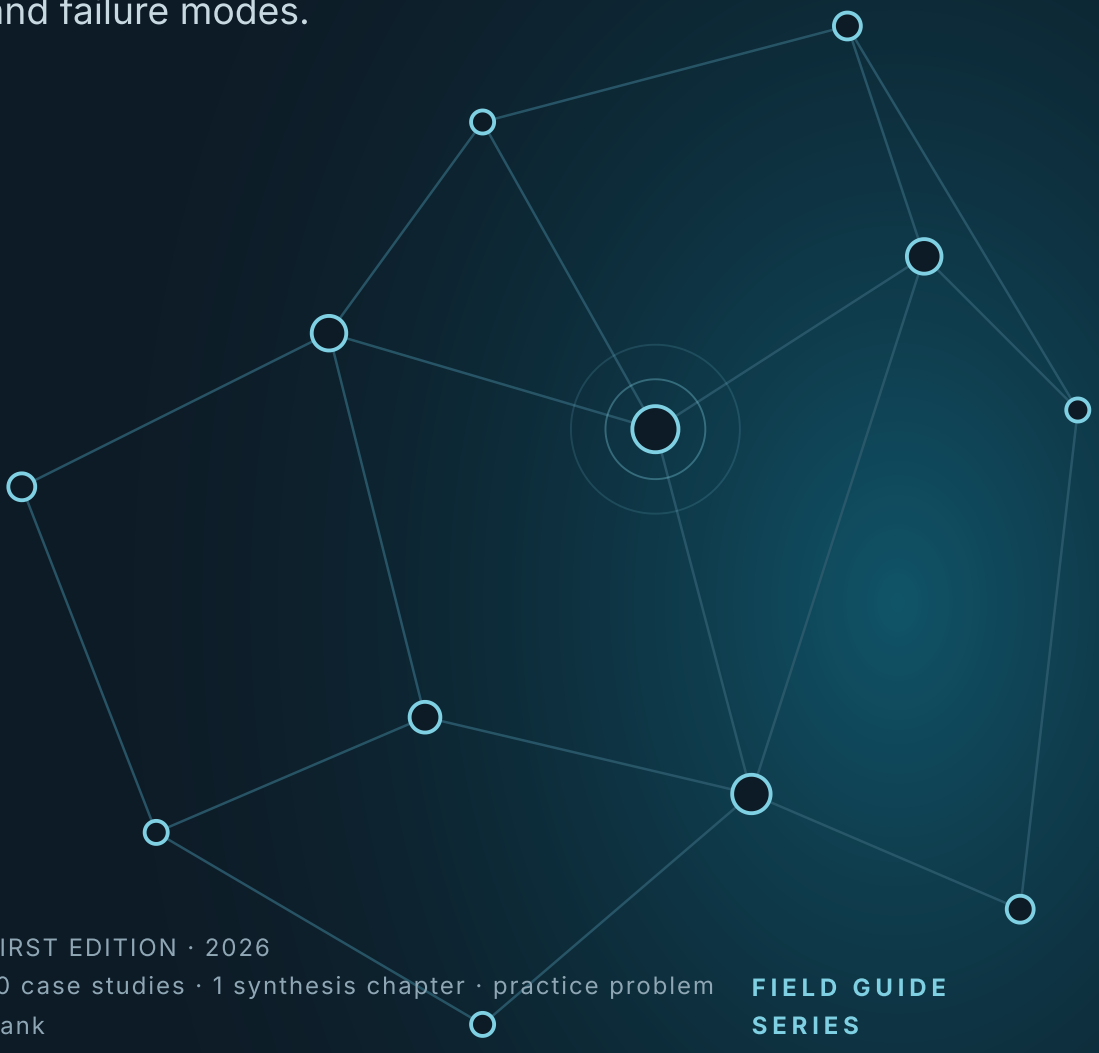
VOLUME 2 · CASE STUDIES

Ten Systems, Designed

Ten real-world systems designed end to end: requirements, numbers, architecture, deep dives and failure modes.



FIRST EDITION · 2026

10 case studies · 1 synthesis chapter · practice problem bank

FIELD GUIDE SERIES

How to use this book

Volume 1 teaches the building blocks. This volume puts them to work.

This PDF is interactive

Everything in the **Contents** is a link. Your PDF viewer's **bookmarks / outline panel** shows the full structure for quick jumps. Every chapter has a ↩ **Contents** button. Every quiz question has a **See answer** → button that jumps to the Answer Key, and every answer has a button that jumps back.

Each chapter follows the same rhythm

Requirements → estimates → API → data model → architecture → deep dives → failure modes → quiz. Try each problem on a blank page before reading it. The boxes mean:

KEY IDEA
The one thing to remember from a section.

TRADE-OFF
What you give up when you choose this.

WORKED EXAMPLE
Numbers, step by step.

INTERVIEW TIP
How to say it under pressure.

COMMON PITFALL
Where real systems go wrong.

GOING DEEPER
Optional detail for the curious.

All text and diagrams in this volume are original. Chapter references such as "Vol. 1, Ch. 12" point to System Design: From First Principles to Planet Scale.

Contents

Preface 4
How each chapter works 4
How to practise with it 4

PART I CONTENT AT SCALE 6

1 Video Streaming Platform 7
1.1 Step 1: Requirements 7
1.2 Step 2: Estimates 7
1.3 Step 3: API 8
1.4 Step 4: Data model 8
1.5 Step 5: High-level design 9
1.6 Step 6: Deep dives 9
1.7 Step 7: Failure modes 11

2 File Storage and Sync 13
2.1 Step 1: Requirements 13
2.2 Step 2: Estimates 13
2.3 Step 3: The core idea: split files into chunks 14
2.4 Step 4: Data model 14
2.5 Step 5: High-level design 15
2.6 Step 6: Deep dives 16
2.7 Step 7: Failure modes 17

3 Web Crawler 18
3.1 Step 1: Requirements 18
3.2 Step 2: Estimates 18
3.3 Step 3: High-level design 19
3.4 Step 4: Deep dives 19
3.5 Step 5: Failure modes 22

PART II CORRECTNESS UNDER PRESSURE 23

4 Payment System 24
4.1 Step 1: Requirements 24
4.2 Step 2: Key concepts 24
4.3 Step 3: API 25
4.4 Step 4: Data model 25
4.5 Step 5: High-level design 26
4.6 Step 6: Deep dives 27
4.7 Step 7: Failure modes 29

5 Ticket Booking System 30

5.1 Step 1: Requirements 30
5.2 Step 2: Estimates 30
5.3 Step 3: Data model 31
5.4 Step 4: High-level design 31
5.5 Step 5: Deep dives 31
5.6 Step 6: Failure modes 34

6 Distributed Rate Limiter 35
6.1 Step 1: Requirements 35
6.2 Step 2: Choosing the algorithm 35
6.3 Step 3: Rules and data 36
6.4 Step 4: High-level design 36
6.5 Step 5: Deep dives 37
6.6 Step 6: Failure modes 38

PART III REAL-TIME EXPERIENCES 40

7 Notification System 41
7.1 Step 1: Requirements 41
7.2 Step 2: API 42
7.3 Step 3: High-level design 42
7.4 Step 4: Deep dives 43
7.5 Step 5: Failure modes 44

8 Collaborative Document Editor 46
8.1 Step 1: Requirements 46
8.2 Step 2: The core problem 46
8.3 Step 3: Operational Transformation (OT) 47
8.4 Step 4: CRDTs 47
8.5 Step 5: High-level design 49
8.6 Step 6: Deep dives 49
8.7 Step 7: Failure modes 50

9 Leaderboards and Live Metrics 52
9.1 Step 1: Requirements 52
9.2 Step 2: The core structure: sorted sets 52
9.3 Step 3: High-level design 53
9.4 Step 4: Deep dives 53
9.5 Step 5: Failure modes 55

10 Search Autocomplete 56
10.1 Step 1: Requirements 56
10.2 Step 2: Core structure: a trie with cached top-K 56
10.3 Step 3: High-level design 57
10.4 Step 4: Deep dives 58
10.5 Step 5: Failure modes 59

PART IV PUTTING IT TOGETHER 61

11 Patterns Across Problems 62

11.1 Pattern 1: Accept fast, process later 62
11.2 Pattern 2: Separate the control plane from the data plane 62
11.3 Pattern 3: Atomic conditional writes guard scarce resources 63
11.4 Pattern 4: Idempotency at every boundary 63
11.5 Pattern 5: Durable log as the source of truth, with derived views 63
11.6 Pattern 6: The outbox for reliable side effects 63
11.7 Pattern 7: Precompute for the read path 64
11.8 Pattern 8: Exact where it matters, approximate everywhere else 64
11.9 Pattern 9: Meter the flood 64
11.10 Pattern 10: One owner per hot object 65
11.11 Pattern 11: Reconcile, don't just trust 65
11.12 A recognition table 66

Practice Problem Bank 68

1. Ride-sharing surge pricing service 68
2. Distributed job scheduler (cron at scale) 68
3. Hotel reservation system 68
4. Metrics monitoring and alerting system 68
5. Typeahead for people search inside a company 69
6. Online auction (eBay-style) 69
7. URL-shortener analytics at 1M clicks/s 69
8. Code deployment system 69
9. Multiplayer game server matchmaking 69
10. Email service (Gmail-style) 70

Answer Key 71

Preface

↩ Contents

Volume 1 taught the building blocks: load balancers, caches, queues, replication, sharding, consensus, sagas, and the arithmetic that tells you which ones you need. This volume puts those blocks together. Each chapter takes one real kind of system and designs it from a blank page to a defensible architecture.

The ten systems were chosen so that, together, they cover the questions that come up again and again in practice: how to move huge files cheaply, how to never lose or double-count money, how to keep many people's views of shared data in sync, how to rank and search at speed, and how to protect a system from its own popularity.

How each chapter works

Every case study follows the same eight steps, so that the method becomes a habit:

1. **Requirements:** functional and non-functional, plus what is out of scope.
2. **Estimates:** the numbers that decide the architecture.
3. **API:** the contract between clients and the system.
4. **Data model:** entities, keys and access patterns.
5. **High-level design:** boxes and arrows, with one request walked end to end.
6. **Deep dives:** the two or three problems that make this system hard.
7. **Failure modes:** what breaks, what users see, and how the system recovers.
8. **Quiz:** questions that test whether the reasoning stuck.

References like **(Vol. 1, Ch. 12)** point back to the chapter of Volume 1 that explains a building block in depth.

How to practise with it

Read only the requirements of a chapter, then close the book. Give yourself 40 minutes and a blank page and design the system yourself, out loud if you can. Then read the chapter and compare. The gaps between your design and the book's are exactly what you need to study.

The practice problem bank at the end gives you ten more prompts for the same exercise, with hints but no full solutions.

PART I

Content at Scale

Systems whose main job is moving and storing very large amounts of data — video, files and the web itself — as cheaply and reliably as possible.

CHAPTER 1

↩ Contents

Video Streaming Platform

Design a platform where creators upload videos and viewers around the world watch them smoothly on any device and connection. The defining traits are enormous file sizes, expensive processing, and a read load that dwarfs everything else.

1.1 Step 1: Requirements

Functional

- Creators upload videos (up to several GB) with a title, description and thumbnail.
- Viewers stream videos on web, mobile and TV, at a quality that adapts to their connection.
- Viewers see view counts, like videos and get a home page of recommendations.
- Search by title and description (we will touch on this only briefly).

Non-functional

- Playback starts quickly (under 2 s) and rarely stalls (rebuffering is the number-one complaint).
- Uploads are never lost once acknowledged. Processing may take minutes.
- Highly available for playback. Brief unavailability of uploads is more tolerable.
- Scale: 500M daily viewers, 1M uploads per day.

Out of scope: live streaming, ads, comments, monetisation.

1.2 Step 2: Estimates

- **Uploads:** 1M/day ≈ 12/s. Average raw upload 500 MB, so about **500 TB/day** of raw video.
- **Transcoded outputs:** about 6 renditions (240p–4K) at a combined ~50% of the raw size, so ~250 TB/day more. Keeping raw files plus renditions with replication or erasure coding means **hundreds of PB per year**. Object storage is the only option.
- **Views:** 500M viewers × 5 videos/day = 2.5B views/day ≈ **29k video starts/s**.

- **Egress:** if an average session streams 300 MB, then 2.5B × 300 MB = 750 PB/day ≈ **70 Tbps average**. No origin can serve this, so nearly all bytes must come from CDN caches close to viewers.

> **KEY IDEA**
>
> **The numbers dictate the design**
>
> Storage in the hundreds of petabytes means object storage with lifecycle tiers. Egress in the tens of terabits means the CDN *is* the delivery system. Upload volume is small by comparison, but each upload triggers heavy, parallel transcoding work.

1.3 Step 3: API

```
POST /v1/uploads                      -> {upload_id, parts: [pre-signed URLs]}
POST /v1/uploads/{id}/complete        {etag_list, title, description}
GET  /v1/videos/{id}                  -> metadata + manifest URL + status
GET  /v1/videos/{id}/manifest.m3u8    -> HLS master playlist (served via CDN)
POST /v1/videos/{id}/views            (batched client heartbeat events)
```

1.4 Step 4: Data model

```
videos(video_id PK, owner_id, title, description, status, duration_s,
       raw_object_key, created_at, published_at)
renditions(video_id, profile, codec, bitrate, segment_prefix, ready_at)
video_stats(video_id PK, views, likes)          -- counters, updated
asynchronously
```

Metadata lives in a sharded relational or wide-column store, sharded by `video_id`. Media lives in object storage: `raw/{video_id}` for the original, and `hls/{video_id}/{profile}/seg_00042.ts` for the segments.

1.5 Step 5: High-level design

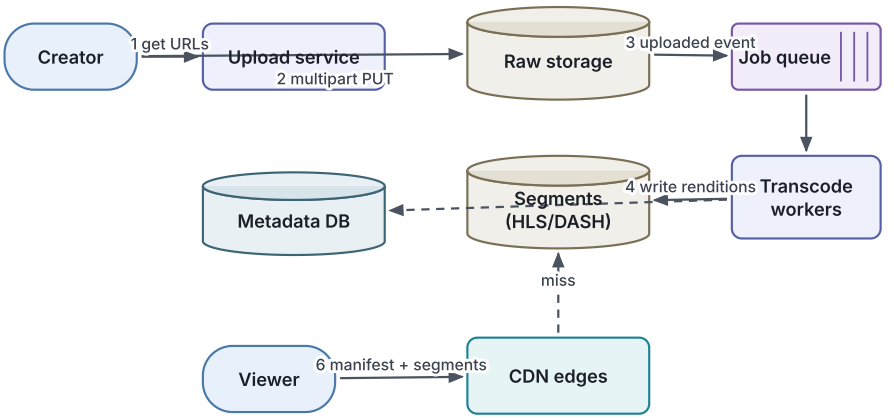


Figure 1.1 Video platform overview. Uploads go straight to object storage, a transcoding pipeline produces segmented renditions, and the CDN serves nearly all playback traffic.

1.6 Step 6: Deep dives

The upload path

Videos are large and connections are flaky, so uploads use **multipart, resumable uploads straight to object storage** with pre-signed URLs (Vol. 1, Ch. 16). The client splits the file into 8–16 MB parts, uploads several in parallel, retries failed parts, and can resume after closing the app. When every part is uploaded, the client calls `complete`. The upload service stitches the parts together, records the video as `UPLOADED`, and emits an event. Application servers never touch the video bytes.

The transcoding pipeline

A single 4K video can take longer than its own duration to encode on one machine. So the pipeline is a **directed acyclic graph (DAG)** of tasks run by a pool of workers:

1. **Validate and probe** the file (container, codec, duration). Reject corrupt files early.
2. **Split** the video into chunks of a few seconds, cut at keyframes.
3. **Encode each chunk × each rendition in parallel**, which may be hundreds of tasks per video, spread across a large worker pool (often spot or preemptible machines, for cost).
4. **Package** the encoded chunks into HLS/DASH segments and write the manifests.
5. **Extract** thumbnails, captions, and fingerprints for copyright matching.
6. **Publish:** mark renditions ready and update metadata.

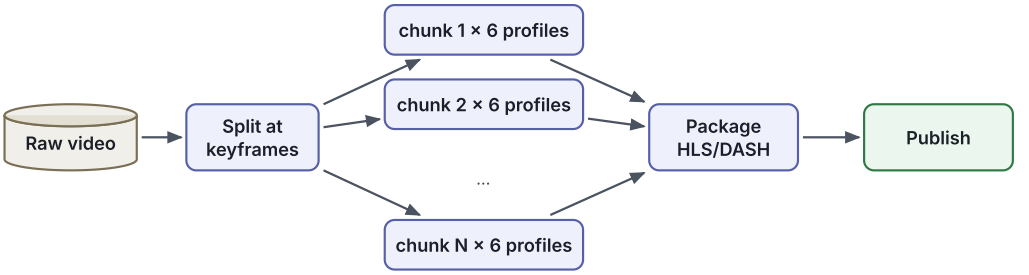


Figure 1.2 Transcoding as a DAG. Chunking turns one slow job into hundreds of small parallel ones.

Tasks are **idempotent** (re-encoding a chunk produces the same output path), so a worker that dies or a preempted machine simply means the task is retried. A workflow engine or a job queue with a state table tracks which tasks of which video have completed.

TRADE-OFF

Encode everything, or encode on demand?

Most videos are watched rarely, and a few are watched enormously. Encoding every upload into every format wastes compute on videos nobody watches. Platforms often encode a few common renditions immediately, then add higher resolutions or more efficient codecs (which are slower to encode) only once a video gets popular.

Adaptive bitrate playback

Each rendition is cut into **segments** of 2–6 seconds. A **master manifest** lists the available renditions, and each rendition's playlist lists its segments. The player measures throughput and buffer level, and picks the rendition for each *next* segment. When bandwidth drops, it switches down instead of stalling. Because segments are small, immutable files with stable URLs, they cache perfectly at the CDN.

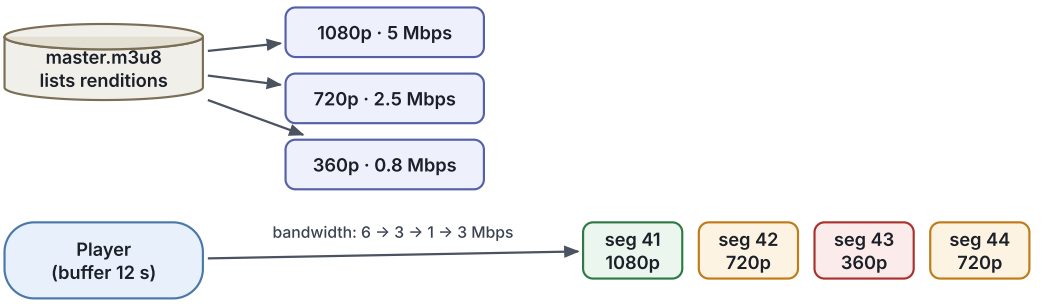


Figure 1.3 Adaptive bitrate. The player picks a rendition per segment from measured bandwidth, stepping down instead of stalling.

CDN strategy

- **Long-tail vs. head:** the most popular few percent of videos account for most views and stay hot in every edge cache. Long-tail videos miss at the edge and are fetched from a **regional mid-tier cache** before the origin (tiered caching), which shields object storage from a flood of misses.
- **Pre-warming:** when a video from a hugely popular creator is published, push its first segments to the edges ahead of demand.
- **Embedded caches:** very large platforms put cache servers inside ISPs' networks, so the traffic never crosses the internet backbone.

View counting

Counting 29k+ views/s exactly, in real time, is unnecessary and expensive. Players send batched heartbeat events to an ingestion endpoint that writes them to a log (Kafka). A stream processor de-duplicates (the same session refreshing the page), filters bots, and aggregates per video per minute, then increments counters in a store. The public count may lag by seconds or minutes. That is fine.

1.7 Step 7: Failure modes

Failure	Effect	Mitigation
Transcode worker dies mid-task	That chunk's task stalls	Task lease expires; another worker retries (idempotent output)
Corrupt or malicious upload	Pipeline errors, possible exploit	Validate in a sandbox; quarantine; mark FAILED and notify the creator
CDN edge outage	Viewers in one city affected	DNS/anycast routes to the next nearest edge
Origin overloaded by misses	Slow starts for long-tail videos	Tiered caches, request coalescing at the mid-tier
Metadata DB unavailable	New pages fail to load	Cache video metadata; playback of already-loaded manifests continues

CHECK YOUR UNDERSTANDING

1. Why do uploads go directly to object storage with pre-signed URLs instead of through the application servers?

See answer →

2. How does chunked transcoding reduce the time to publish a long video?

   See answer →

3. What makes video segments ideal for CDN caching?

   See answer →

4. Why are view counts aggregated asynchronously instead of incremented in the database on every view?

   See answer →

CHAPTER 2

↩ Contents

File Storage and Sync

Design a service like Dropbox or Google Drive: users keep files in a folder on each of their devices, and changes on one device appear on all the others. Files can be shared with other users.

2.1 Step 1: Requirements

Functional

- Upload, download, rename, move and delete files and folders.
- Automatic sync across a user's devices (desktop, mobile, web).
- Share files and folders with other users (view or edit).
- Version history: restore earlier versions of a file.

Non-functional

- **Never lose data.** Durability is the top priority.
- Sync changes within seconds when online. Work offline and reconcile later.
- Bandwidth-efficient: a small edit to a large file shouldn't re-upload the whole file.
- Scale: 100M users, 1B files, 10 PB stored.

2.2 Step 2: Estimates

- 100M users × ~100 files modified per week ≈ 10B changes/week ≈ **16k file changes/s**.
- Average file 1 MB, but sizes are long-tailed: many tiny files, some multi-GB.
- Storage: 10 PB of unique content, and *much* more without deduplication.
- Metadata: 1B files × ~1 KB ≈ 1 TB, plus version history. That is a modest amount in bytes, but it gets heavy read/write traffic.

2.3 Step 3: The core idea: split files into chunks

Instead of treating a file as one blob, the client splits it into **chunks** (for example, 4 MB each) and computes a cryptographic hash of each chunk. A file is then just an ordered list of chunk hashes, its **manifest**.

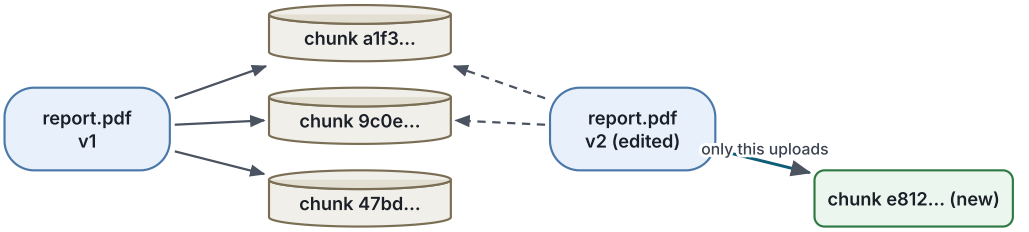


Figure 2.1 A file as a list of content-addressed chunks. Editing one part of a file changes only the chunk that contains it.

This one idea delivers three big wins:

- **Delta sync:** when a file changes, only the chunks that changed are uploaded.
- **Deduplication:** chunks are stored by hash (*content-addressed*). If any user anywhere has already stored that chunk, it is never stored or uploaded again.
- **Resumable transfers:** an interrupted upload resumes at the next missing chunk.

> GOING DEEPER
>
> **Content-defined chunking**
>
> With fixed-size chunks, inserting one byte at the start of a file shifts every chunk boundary, so every chunk hash changes. **Content-defined chunking** places boundaries where a rolling hash of the data matches a pattern, so the boundaries move with the content and an insertion changes only the chunks around it. Backup and sync tools use this to make dedup and deltas robust to edits in the middle of files.

2.4 Step 4: Data model

```
files(file_id PK, owner_ns, parent_id, name, is_folder, latest_version,
deleted)
versions(file_id, version, chunk_list[hash...], size, modified_by, modified_at)
chunks(hash PK, size, storage_location, ref_count)
namespace_journal(namespace_id, seq, op, file_id, version, ts)   -- ordered
change log
```

A **namespace** is a user's root folder or a shared folder. Each namespace has an append-only, ordered **journal** of changes. This journal is the heart of sync.

2.5 Step 5: High-level design

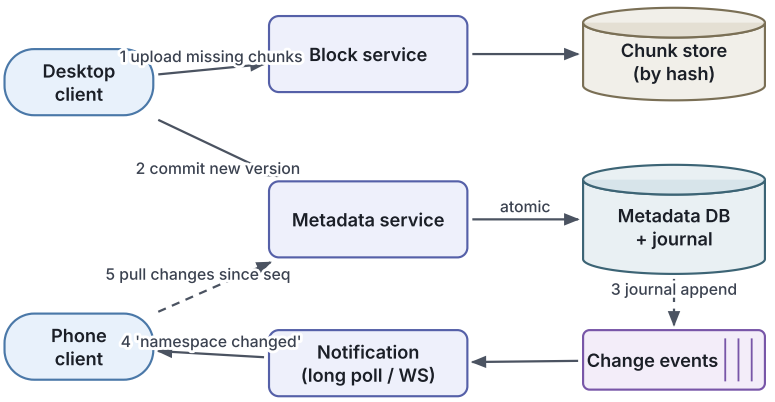


Figure 2.2 File sync architecture. Chunk bytes go to block storage by hash, metadata commits go to a transactional store, and a notification service tells other devices to pull.

Uploading a change:

1. The client detects a file change (via the OS file watcher), re-chunks the file and hashes the chunks.
2. It asks the block service which hashes are missing, and uploads only those.
3. It **commits** the new version to the metadata service: "file X, base version 7, new chunk list [...]". The metadata service checks the base version (optimistic concurrency), writes the new version and appends to the namespace journal, **in one transaction**.
4. Other devices subscribed to that namespace get a notification and **pull** all journal entries after the last sequence number they have seen, then download any chunks they lack.

> **KEY IDEA**
>
> **Separate the control plane from the data plane**
>
> Small, critical, transactional metadata goes through the metadata service. Huge, immutable chunk bytes go through the block service to object storage. Each can be scaled, cached and secured on its own terms.

2.6 Step 6: Deep dives

Conflicts

Two devices edit the same file while one is offline. When the second device commits with a base version that is no longer current, the metadata service rejects the commit. The client then keeps both versions: it saves its own as `report (conflicted copy from Laptop, 2026-10-15).pdf` and syncs the server's version. Automatically merging arbitrary binary files is impossible, and keeping both never loses data. (Collaborative editors take a different approach; see Chapter 8.)

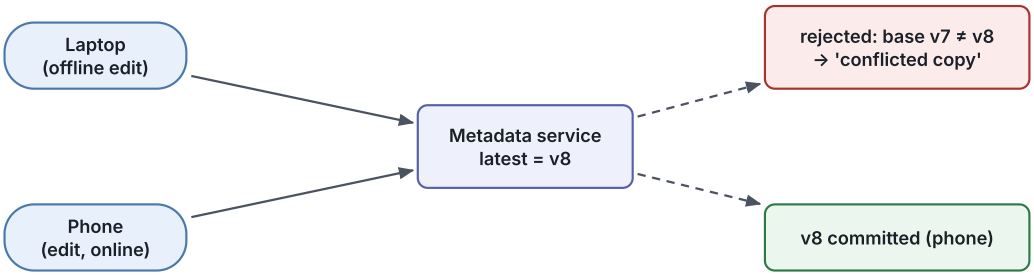


Figure 2.3 Conflict handling. The second commit fails the base-version check, and the client keeps its edit as a conflicted copy.

Notifications

Millions of idle desktop clients need to hear about changes within seconds. Each keeps a long-poll or WebSocket connection to the notification service (Vol. 1, Ch. 10), subscribed to its namespaces. Notifications carry no data, only "namespace N changed". The client then pulls from the journal. If a notification is lost, nothing breaks: clients also poll occasionally, and the journal is the source of truth.

Metadata scaling

The metadata store needs transactions (version checks, journal appends, moving folders) and heavy read traffic. Shard it by **namespace**, so that every operation inside a folder tree stays in one shard. Shared folders are their own namespaces, mounted into the trees of each member. Listing a folder and syncing are both reads within one namespace.

Deleting data safely

Deleting a file just writes a new journal entry (a tombstone). Chunks are shared between files and users, so a chunk can only be removed when no version anywhere refers to it. Use **reference counting** or periodic **mark-and-sweep** garbage collection, with a grace period and

version-history retention (for example, 30 days), so that accidental deletes can be undone.

Security and privacy

Encrypt chunks at rest (Vol. 1, Ch. 25). Cross-user deduplication can leak information: an attacker can learn whether someone already has a particular file by watching whether its upload is skipped. Many services therefore deduplicate only within a user's own data, or within an organisation.

2.7 Step 7: Failure modes

Failure	Effect	Mitigation
Upload interrupted	Partial file	Chunks resume; commit only after all chunks exist
Commit races from two devices	Conflict	Optimistic version check; conflicted copy
Missed notification	Delayed sync	Periodic polling; journal is the source of truth
Chunk store region down	Downloads fail	Erasure-coded, multi-region replication of chunks
Bug deletes metadata	Data appears lost	Journals, version history, point-in-time backups

CHECK YOUR UNDERSTANDING

1. List three benefits of splitting files into content-addressed chunks.

   See answer →

2. Why does the sync design use a per-namespace journal with sequence numbers?

   See answer →

3. Two devices edit the same file offline. What happens when they reconnect, and why not merge automatically?

   See answer →

4. A 2 GB video file has one paragraph's worth of frames changed at the start. Why does content-defined chunking matter here?

   See answer →

5. Why is metadata sharded by namespace rather than by file ID?

   See answer →

CHAPTER 3

↩ Contents

Web Crawler

Design a crawler that downloads billions of web pages for a search engine: it starts from seed URLs, fetches pages, extracts links, and keeps going, while being polite to websites and avoiding traps.

3.1 Step 1: Requirements

Functional

- Start from seed URLs, fetch pages, parse them, extract links, and add new links to the crawl.
- Store page content for the indexer.
- Re-crawl pages periodically, more often if they change often.

Non-functional

- **Politeness:** never overload a website. Respect `robots.txt` and crawl delays.
- **Scale:** 5B pages per month.
- **Robustness:** survive malformed HTML, infinite URL spaces (crawler traps), slow servers and duplicate content.
- **Extensibility:** new content types and processing steps can be added later.

3.2 Step 2: Estimates

- 5B pages/month ÷ 2.6M s ≈ **2,000 pages/s** on average.
- Average page ~100 KB, so ~200 MB/s download, and about **500 TB/month** of raw HTML (much less compressed).
- URLs seen: tens of billions over time. At ~100 bytes each, a full set is several TB, too big for one machine's memory, which is a hint that compact, approximate structures will be useful.

3.3 Step 3: High-level design

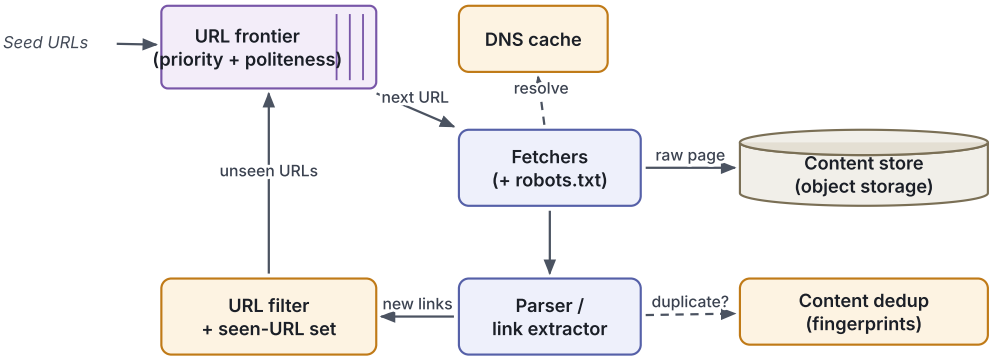


Figure 3.1 The crawl loop. The frontier decides what to fetch next, fetchers download politely, and parsed links flow back through de-duplication filters.

The loop: the frontier hands a URL to a fetcher. The fetcher checks `robots.txt` (cached per host), resolves DNS (cached), downloads the page, and stores it. The parser extracts links and normalises them (lowercase host, remove fragments and tracking parameters). The links are filtered against the set of URLs already seen, and new ones go back into the frontier.

3.4 Step 4: Deep dives

The URL frontier: priority and politeness

The frontier must answer two questions at once: *which URL matters most?* and *which host is it safe to hit right now?*

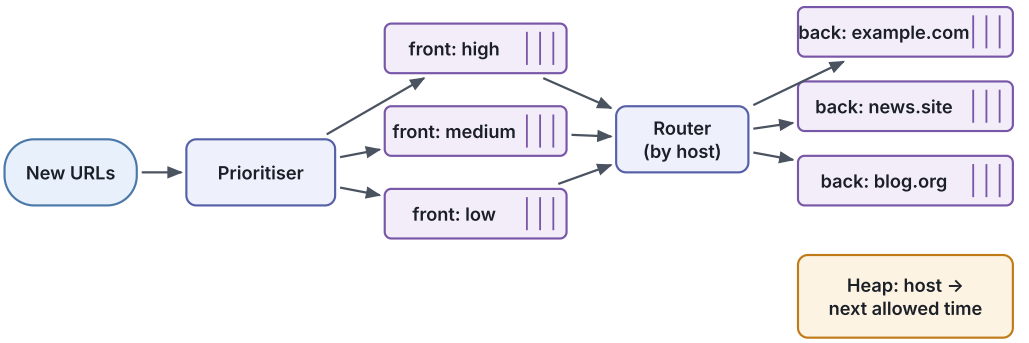


Figure 3.2 Frontier design. Front queues order URLs by priority, and back queues hold one host each so that each host is fetched at a polite rate.

- **Front queues (priority):** URLs are scored by importance (link-based page rank, domain authority, how often the page changes) and placed into priority queues. Higher-priority queues are drained more often.
- **Back queues (politeness):** each back queue holds URLs for **a single host**. A heap records each host's **next allowed fetch time**. A fetcher takes the host whose time has come, fetches one URL, and pushes that host's next time forward by its crawl delay (for example, 1 s, or several times the last response time).

The frontier is far too big for memory, so it is stored on disk (in a log or a database), with in-memory buffers at the heads and tails of the queues. It is partitioned across crawler nodes **by host** (consistent hashing on the hostname), so each host's politeness is enforced by exactly one node.

"Have we seen this URL?"

Checking tens of billions of URLs against a database for every extracted link is too slow. Use a **Bloom filter** (Vol. 1, Ch. 21) over normalised URL hashes: "definitely new" URLs go straight into the frontier, and "probably seen" ones are dropped. A small false-positive rate means a few new pages are skipped, which is acceptable for a crawler. Partition the filter by host along with the frontier.

Duplicate content

Many URLs serve the same or almost the same content (mirrors, session IDs in URLs, printer-friendly versions). Exact duplicates are caught with a content hash. **Near-duplicates** are caught with similarity fingerprints such as **SimHash** or MinHash. Similar documents get fingerprints that differ in only a few bits, which can be found quickly. Duplicates are stored once, and their links aren't followed again.

Crawler traps and robustness

- **Infinite URL spaces:** calendars with a "next month" link forever, or session IDs generating endless unique URLs. Defend with maximum URL length, maximum depth per host, per-host page budgets, and URL pattern detection.
- **Slow or hostile servers:** strict connect and read timeouts, maximum page size, and limited redirects.
- **Malformed content:** a fault-tolerant parser, run in isolated workers.

Freshness: re-crawling

Pages change at very different rates: a news front page every minute, an old blog post never. Estimate each page's change rate from its history (did its content hash change since the last crawl?), and schedule re-crawls with a frequency proportional to change rate × importance. Conditional requests (`If-Modified-Since`, `ETag`) make unchanged pages cheap to re-check.



Figure 3.3 Re-crawl scheduling. Pages that change often and matter more are revisited sooner.

DNS

At 2,000 fetches/s, DNS lookups would become a bottleneck and annoy resolvers. Run local caching resolvers, cache per host for the record's TTL, and resolve upcoming hosts asynchronously ahead of time.

3.5 Step 5: Failure modes

Failure	Effect	Mitigation
Crawler node dies	Its hosts stop being crawled	Frontier on durable storage; reassign host partitions via consistent hashing
Host overloaded by us	Site blocks us, complaints	Per-host back queues, adaptive delays, honour robots.txt and 429/503
Trap generates endless URLs	Frontier floods with junk	Depth limits, per-host budgets, pattern detection
Parser crash on bad page	Worker restarts	Isolate parsing; skip and log the page

CHECK YOUR UNDERSTANDING

1. Why does the frontier use separate front (priority) and back (per-host) queues?

   See answer →

2. Why is a Bloom filter a good fit for the seen-URL check, and what is the cost of its false positives here?

   See answer →

3. How can a crawler avoid getting stuck in a calendar that links to "next month" forever?

   See answer →

4. How does a crawler detect that two different URLs serve nearly identical pages?

   See answer →

5. Why is the frontier partitioned by hostname across crawler nodes?

   See answer →

PART II

Correctness Under Pressure

Systems where a single mistake costs real money or trust — double charges, oversold seats, unfair limits — and where correctness must survive retries, races and traffic spikes.

CHAPTER 4

↩ Contents

Payment System

Design the payment backend of an online marketplace: buyers pay for orders with cards and wallets, the platform keeps a fee, and sellers are paid out. Money must never be lost, created or moved twice.

4.1 Step 1: Requirements

Functional

- Accept payments for orders through external payment service providers (PSPs) that handle the card networks.
- Refunds, full and partial.
- Track balances for the platform and sellers. Pay sellers out on a schedule.
- A complete, auditable history of every money movement.

Non-functional

- **Correctness above all:** no double charges, no lost payments, every cent accounted for.
- **Idempotent** under retries at every layer.
- High availability for checkout, while staying strongly consistent for balances.
- Moderate throughput: 1,000 payments/s at peak. Payment systems are rarely limited by raw scale.
- Security and compliance: we must never store raw card numbers ourselves (PCI-DSS scope).

4.2 Step 2: Key concepts

- **PSP (payment service provider):** an external company (a card processor, a wallet provider) that actually moves money through card networks and banks. We call its API. It calls us back with **webhooks**.

- **Authorisation vs. capture:** an *authorisation* reserves funds on the buyer's card. A *capture* actually takes them (often when the order ships). Uncaptured authorisations expire.
- **Tokenisation:** the buyer's card details go straight from their browser to the PSP's hosted form or SDK, which returns a **token**. Our servers only ever see the token, which keeps us out of most PCI scope.
- **Ledger:** our internal, append-only record of every money movement, using **double-entry bookkeeping**.

4.3 Step 3: API

```
POST /v1/payments
  Idempotency-Key: 7c1e…
  {order_id, amount: 4999, currency: "EUR", payment_token, buyer_id}
  -> 201 {payment_id, status: "PENDING" | "SUCCEEDED" | "FAILED" |
"REQUIRES_ACTION"}

POST /v1/payments/{id}/refunds   Idempotency-Key: …  {amount}
POST /v1/webhooks/psp            (signed callbacks from the PSP)
GET  /v1/payments/{id}
```

Amounts are **integers in minor units** (cents). Never use floating point for money.

4.4 Step 4: Data model

```
payments(payment_id PK, order_id, buyer_id, amount, currency, status,
         psp, psp_reference, idempotency_key UNIQUE, created_at, updated_at)
payment_events(payment_id, seq, from_status, to_status, payload, ts)    --
audit trail
ledger_entries(entry_id PK, txn_id, account_id, direction(DEBIT/CREDIT),
               amount, currency, created_at)                             --
append-only
accounts(account_id PK,
type(buyer_clearing/seller/platform_fees/psp_receivable), currency)
```

Double-entry bookkeeping

Every movement of money is a **transaction made of at least two entries whose debits and credits sum to zero**. Money is never created or destroyed. It only moves between accounts. A payment of €49.99 with a 10% platform fee becomes:

Account	Debit	Credit
PSP receivable (money the PSP owes us)	49.99	
Seller balance		44.99
Platform fees		5.00
Total	**49.99**	**49.99**

Balances are computed by summing entries (with periodic snapshots for speed). The ledger is **append-only**. Mistakes are corrected with new, reversing entries, never by editing old ones, so the history is a complete audit trail.

> **KEY IDEA**
>
> **The ledger is the source of truth for money**
>
> Statuses on payment rows tell you where a workflow is. The double-entry ledger tells you where the money is. Because every transaction must balance, a whole class of bugs (money appearing or vanishing) becomes detectable with a single sum.

4.5 Step 5: High-level design

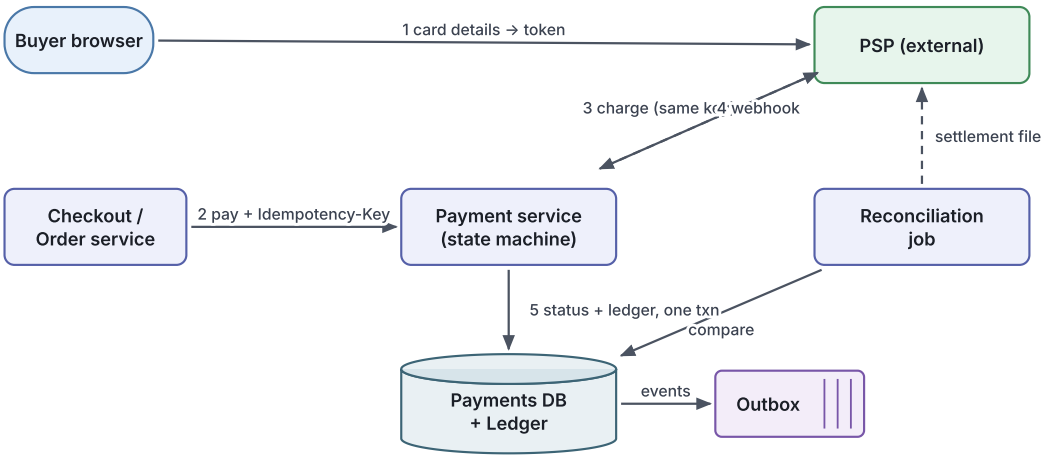


Figure 4.1 Payment flow. The payment service drives a state machine, calls the PSP with idempotency keys, records ledger entries atomically, and reconciles against the PSP's reports.

4.6 Step 6: Deep dives

Idempotency end to end

Retries happen at every hop: the browser double-clicks, the checkout service times out, the payment service crashes after calling the PSP. Correctness relies on **idempotency at each boundary**:

1. **Client → payment service:** the `Idempotency-Key` header (derived from `order_id` plus attempt) is stored with a unique constraint. A repeated request returns the stored result instead of starting a second payment.
2. **Payment service → PSP:** we pass our own payment ID as the PSP's idempotency key, so a retried charge after a timeout returns the original result instead of charging again.
3. **PSP → us (webhooks):** webhooks may arrive late, out of order, or more than once. We verify the signature, look up the payment by `psp_reference`, and apply the transition only if it is valid from the current state (`PENDING → SUCCEEDED`, never `FAILED → SUCCEEDED` without a new attempt), recording the event ID so that duplicates are ignored.

The payment state machine

```
CREATED → PENDING ──→ SUCCEEDED → (PARTIALLY_)REFUNDED
            │   └──→ REQUIRES_ACTION (3-D Secure) → PENDING
            └──────→ FAILED
```

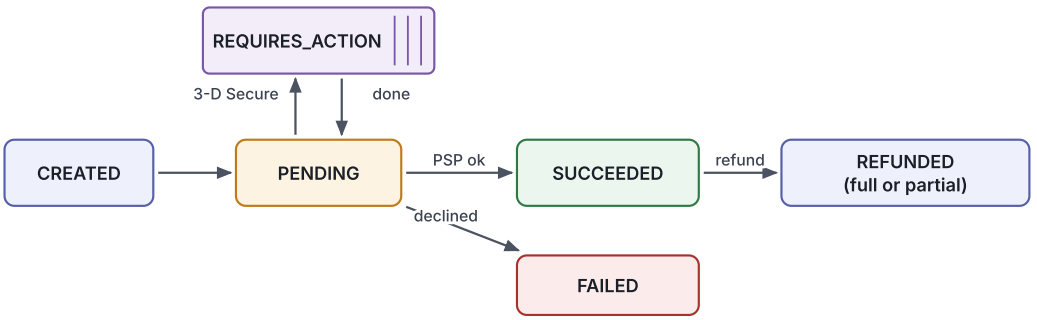


Figure 4.2 The payment state machine. Each arrow is a guarded, conditional update; anything else is rejected.

Every transition is a **conditional update** (`UPDATE … SET status='SUCCEEDED' WHERE id=? AND status='PENDING'`) written **in the same database transaction** as its ledger entries and an outbox event (Vol. 1, Ch. 20). So a payment can never be marked as succeeded without the matching ledger entries, and downstream systems (order fulfilment, emails) are notified reliably, at least once.

The unknown outcome

The hardest case: we call the PSP and the request **times out**. Did the charge happen? We can't know (Vol. 1, Ch. 19). The wrong reactions are to assume failure (the customer pays again and is charged twice) or to assume success (we ship without payment). The right reaction is to leave the payment in `PENDING` (or `UNKNOWN`), then resolve it by **querying the PSP** for that idempotency key, by waiting for the webhook, and finally through reconciliation. Retrying the charge with the same idempotency key is also safe.

Reconciliation

Distributed systems drift: webhooks get lost, bugs happen, and PSPs make mistakes. Every day, the PSP provides a **settlement report** listing every transaction it processed. A reconciliation job matches it line by line against our payments and ledger:

- Matched: good.
- In the PSP's report but not ours: a missed webhook or a crash. Fix our state.
- In ours but not the PSP's: a charge we think happened but didn't. Investigate.
- Amount or currency mismatch: raise an alert, and correct with reversing entries.

Reconciliation is the safety net that turns "we hope it's right" into "we prove it's right" every day.

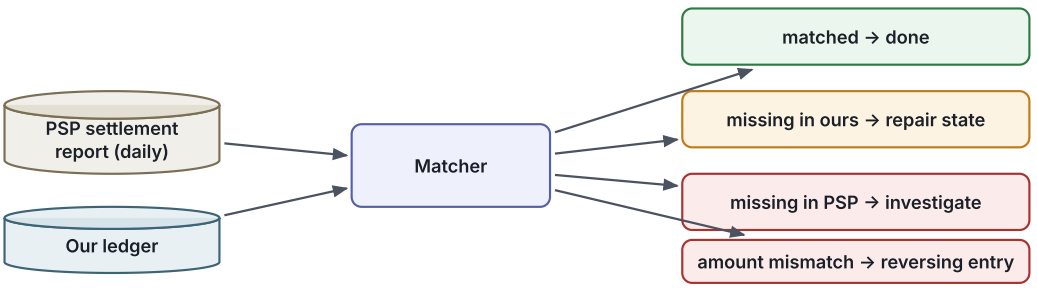


Figure 4.3 Daily reconciliation. Every line of the PSP's settlement report is matched against the internal ledger.

Payouts and refunds

Payouts aggregate each seller's ledger balance and send transfers through a payout provider, recorded as ledger transactions (`seller balance` debit, `bank clearing` credit) with their own idempotency keys and reconciliation. Refunds are new transactions that reverse the original split, never edits of it.

Consistency and scale

Throughput is modest, so a **strongly consistent relational database** with serializable or carefully locked transactions is the right tool. Shard by `merchant_id` or `account_id` if needed, keeping each ledger transaction within one shard where possible, or use a ledger database built for this. Hot accounts (the platform's fee account receives a credit on every payment) can be split into several sub-accounts that are summed when read.

4.7 Step 7: Failure modes

Failure	Effect	Mitigation
Double-click / client retry	Duplicate request	Idempotency key with unique constraint
PSP timeout	Outcome unknown	Keep PENDING; query PSP; safe retry with same key; reconcile
Duplicate / out-of-order webhook	Wrong state transition	Signature check, event de-dup, state-machine guards
Crash between DB write and event publish	Fulfilment never triggered	Transactional outbox
Silent drift between us and PSP	Money mismatch	Daily reconciliation, alerting, reversing entries

CHECK YOUR UNDERSTANDING

1. Why should amounts be stored as integers in minor units rather than floating point?
   See answer →
2. What is double-entry bookkeeping, and how does it help catch bugs?
   See answer →
3. A call to the PSP times out. Why is it wrong to mark the payment as failed, and what should you do?
   See answer →
4. A webhook saying "payment succeeded" arrives twice, and another saying "failed" arrives after the success. How should the service handle them?
   See answer →
5. What does daily reconciliation catch that idempotency and webhooks don't?
   See answer →

CHAPTER 5

↩ Contents

Ticket Booking System

Design a system that sells seats for concerts and sports events. Most of the time traffic is modest, but when a popular event goes on sale, millions of people arrive in the same minute for a few tens of thousands of seats.

5.1 Step 1: Requirements

Functional

- Browse events and see a seat map with live availability.
- Select seats and hold them for a few minutes while paying.
- Complete purchase. Seats become sold and tickets are issued.
- Release holds that expire or are abandoned.

Non-functional

- **No double-selling:** a seat is sold at most once. This is non-negotiable.
- **Fairness:** during on-sales, users get in roughly in arrival order, and bots don't win.
- **Survive spikes:** 10M users arriving within minutes for 50k seats.
- Seat maps may be slightly stale while browsing. The final booking must be exact.

5.2 Step 2: Estimates

- Normal day: about 100 bookings/s across all events.
- Hot on-sale: 10M users, many refreshing the seat map every few seconds, which means **millions of reads/s** for one event, and tens of thousands of hold attempts per second on the same 50k rows.
- The data is small (an event has tens of thousands of seat rows). The problem is **contention**, not volume.

KEY IDEA

Contention, not size

A stadium's seats fit in memory on one machine. What makes this hard is that millions of requests compete for the same few thousand rows in the same seconds. The design is about controlling that competition fairly and safely.

5.3 Step 3: Data model

```
events(event_id PK, venue_id, name, starts_at, on_sale_at, status)
seats(event_id, seat_id, section, row, number, price_tier,
      status(AVAILABLE/HELD/SOLD), hold_id, hold_expires_at, version)
      PRIMARY KEY (event_id, seat_id)
holds(hold_id PK, user_id, event_id, seat_ids[], expires_at, status)
orders(order_id PK, hold_id UNIQUE, user_id, amount, status, payment_id)
```

5.4 Step 4: High-level design

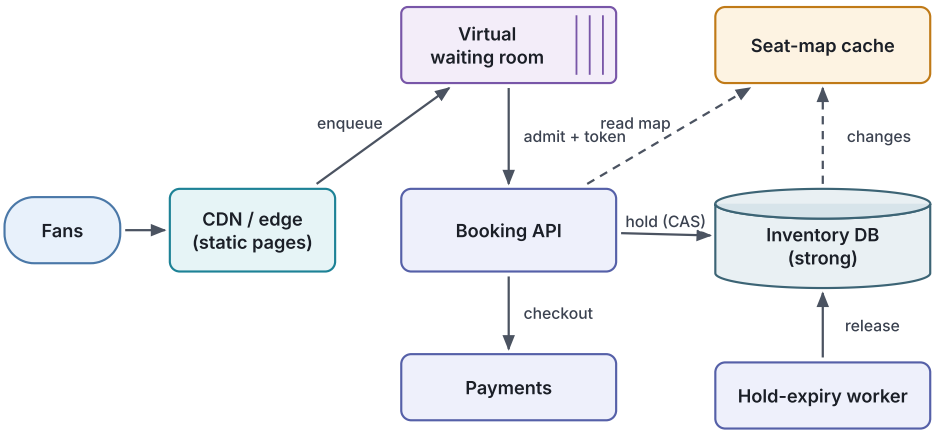


Figure 5.1 Ticketing architecture. A virtual waiting room meters users into the booking path, seat-map reads come from a cache, and holds are taken with atomic conditional writes.

5.5 Step 5: Deep dives

Holding seats without double-selling

When a user selects seats, the booking service tries to move them from AVAILABLE to HELD **atomically**, in one statement or transaction:

```
UPDATE seats
   SET status = 'HELD', hold_id = :h, hold_expires_at = now() + interval '8
minutes'
 WHERE event_id = :e AND seat_id IN (:s1, :s2)
   AND status = 'AVAILABLE';
-- succeed only if rows_affected == 2, else roll back and report "seat taken"
```

Only one transaction can win each seat: the `WHERE status = 'AVAILABLE'` condition is a compare-and-set. Everyone else gets "seat no longer available" and picks again. Taking all of a user's seats in one transaction (or all-or-nothing with rollback) prevents half-held groups.

Purchase then moves seats from HELD to SOLD *only if the hold is still theirs and hasn't expired* (`WHERE hold_id = :h AND hold_expires_at > now()`), in the same transaction as creating the order, which is linked to a payment idempotency key (Chapter 4).

TRADE-OFF

Pessimistic vs. optimistic locking

`SELECT … FOR UPDATE` locks the rows while the transaction runs (pessimistic). That is simple, but under heavy contention, transactions queue up behind locks. Conditional updates and version columns (optimistic) never wait. Losers fail fast and retry. With many users competing for few seats, failing fast and asking the user to choose again gives better throughput and a clearer experience.

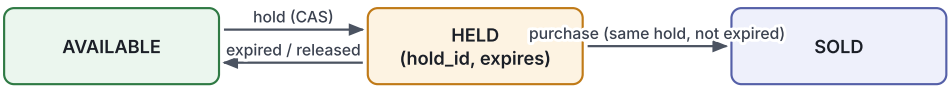


Figure 5.2 Seat lifecycle. Each transition is an atomic conditional write, and holds fall back to AVAILABLE when they expire.

Expiring holds

Holds must expire so that abandoned carts return seats to sale. Two complementary mechanisms:

- **Lazy:** the hold query treats `HELD` seats whose `hold_expires_at` has passed as available (`WHERE status='AVAILABLE' OR (status='HELD' AND hold_expires_at < now())`).

- **Active:** a background worker regularly releases expired holds and updates the seat-map cache, so that browsing users see the seats come back.

Using *time-bounded holds* is the same idea as leases (Vol. 1, Ch. 19): ownership that disappears on its own if the owner vanishes.

The virtual waiting room

Letting 10M users hit the booking API at once would melt it, and it would reward whoever has the fastest connection or the most bots. Instead, during an on-sale:

1. Users arriving before or at on-sale time get a place in a **queue**, either randomised among early arrivals (so being first by milliseconds doesn't matter) or in arrival order.
2. The waiting-room page is static, served from the CDN, and polls a lightweight queue-position endpoint.
3. An **admission controller** lets users in at a rate the booking system can handle (for example, 2,000 per minute), issuing each a short-lived **signed token**.
4. The booking API accepts only requests carrying a valid admission token. Everyone else stays in line.



Figure 5.3 The waiting room turns an uncontrollable spike into a steady, fair flow.

Combined with bot defences (CAPTCHA at entry, device fingerprinting, per-account purchase limits), this keeps the system up and the sale fair.

Serving seat maps

Millions of fans refreshing a seat map must not query the inventory database. The map is a **read model** in a cache, updated from inventory changes (CDC or events) and pushed to the CDN or to clients every second or two. It is allowed to be slightly stale: a seat shown as available may already be held, and the atomic hold catches that. For the hottest sales, show availability by section ("few left") rather than seat by seat, which reduces both load and wasted clicks.

Scaling inventory writes

A single event's seats live in one partition of the inventory store, partitioned by `event_id`, so each hold is a single-partition transaction. For extreme events, split the inventory into **sections**, each owned by its own partition or in-memory actor. A booking for seats in section 112 touches only that section's owner.

5.6 Step 6: Failure modes

Failure	Effect	Mitigation
Two users click the same seat	Race	Conditional update; exactly one wins
User abandons checkout	Seats stuck	Hold expiry (lazy + active)
Payment succeeds after hold expired	Paid but seat released	Extend hold during payment; if lost, refund automatically or offer alternatives
On-sale spike	API overload, unfairness	Waiting room, admission tokens, CDN-served pages
Bots	Real fans lose out	CAPTCHA, purchase limits, anomaly detection

CHECK YOUR UNDERSTANDING

1. How does the system guarantee a seat is never sold twice?

   See answer →

2. Why use time-bounded holds instead of locking seats until the user finishes?

   See answer →

3. What problems does a virtual waiting room solve during a big on-sale?

   See answer →

4. Why is it acceptable for the seat map to be slightly stale?

   See answer →

CHAPTER 6

↩ Contents

Distributed Rate Limiter

Design a rate-limiting service used by an API platform with gateways in several regions. It must enforce limits such as "1,000 requests per minute per API key" and "5 login attempts per minute per account", while adding almost no latency.

6.1 Step 1: Requirements

Functional

- Define rules by key type (API key, user, IP, endpoint), limit and window, plus tiers (free and paid).
- On each request, decide *allow* or *reject* (HTTP 429 with `Retry-After`).
- Return headers showing the limit, how much remains and when it resets.

Non-functional

- **Low latency:** under 1–2 ms added at p99. The limiter sits on every request.
- **High availability:** limiter trouble must not take down the API.
- **Accuracy:** close enough. Small overshoots are acceptable for most rules, but security rules (login) should be strict.
- Scale: 1M requests/s across 50 gateway instances in 3 regions, 10M active keys.

6.2 Step 2: Choosing the algorithm

Vol. 1, Ch. 9 compared the algorithms. For a general-purpose API limiter, two stand out:

- **Token bucket:** allows bursts up to the bucket size, enforces an average rate, and needs only two numbers per key (`tokens`, `last_refill`).
- **Sliding window counter:** two counters per key, smooth limits, no boundary bursts.

We'll use a **token bucket** by default and sliding windows for rules that need smoothness.

6.3 Step 3: Rules and data

```
- match: {key_type: api_key, tier: free}
  limit: 1000 per 60s, burst: 100
- match: {key_type: account, endpoint: POST /login}
  limit: 5 per 60s, mode: strict
```

Rules are stored centrally, pushed to every gateway, and cached in memory. Live counters are stored as `rl:{rule}:{key}` → `{tokens, ts}` with a TTL slightly longer than the window, so idle keys disappear on their own.

6.4 Step 4: High-level design

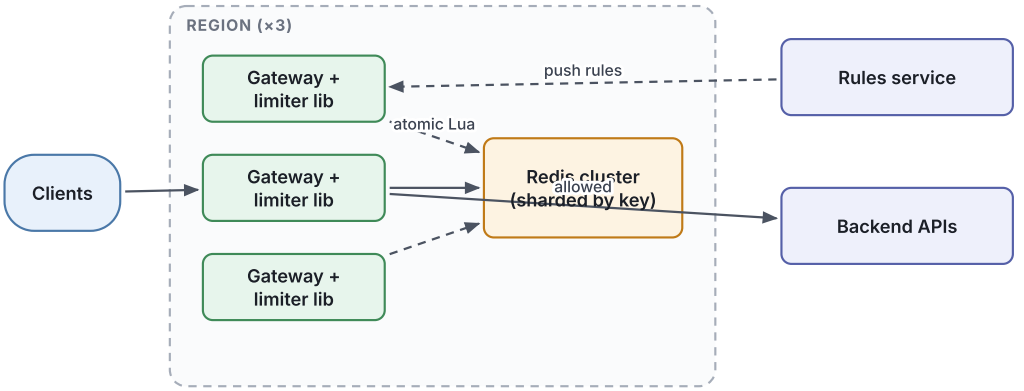


Figure 6.1 Rate limiter architecture. Gateways check a local fast path, consult a regional Redis cluster atomically, and fall back to local limits if Redis is unavailable.

Request path: the gateway's limiter library finds the matching rules, then executes **one atomic script** in Redis that refills and takes a token for each rule key, returning allow/deny plus the remaining count. The Redis cluster is sharded by key (consistent hashing), so the load spreads out and each key's state lives on one shard.

```
-- KEYS[1]=bucket key; ARGV: capacity, refill_per_ms, now_ms, cost
local b = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens = tonumber(b[1]) or tonumber(ARGV[1])
local ts = tonumber(b[2]) or tonumber(ARGV[3])
tokens = math.min(tonumber(ARGV[1]), tokens + (ARGV[3] - ts) * ARGV[2])
local allowed = tokens >= tonumber(ARGV[4])
if allowed then tokens = tokens - ARGV[4] end
redis.call('HSET', KEYS[1], 'tokens', tokens, 'ts', ARGV[3])
redis.call('PEXPIRE', KEYS[1], 120000)
return {allowed and 1 or 0, tokens}
```

Running the read-modify-write as one script makes it atomic: two gateways can't both spend the last token.

6.5 Step 5: Deep dives

Latency: avoiding a network hop on every request

A Redis round trip inside the region costs ~0.3–1 ms, which is usually acceptable. To go further:

- **Local pre-check:** keys that are clearly far below their limit (from recent responses) can be allowed locally, with usage reported asynchronously in batches. Only keys near their limit check Redis synchronously.
- **Local deny cache:** once a key is denied, cache "denied until T" locally, so abusive clients don't generate Redis traffic at all.
- **Pipelining:** check all of a request's rules in one round trip.

Multiple regions

Clients may hit any region. Options:

1. **Per-region limits:** each region enforces limit ÷ number of regions, or a share weighted by traffic. Simple and fast, but a client concentrated in one region gets less than its full limit.
2. **Global counters, asynchronously synced:** each region counts locally and periodically exchanges deltas with the others (a G-counter CRDT per window, Vol. 1, Ch. 14). It is approximately global, and the overshoot is bounded by the sync interval.
3. **Home region per key:** route each key's checks to one region. Exact, but cross-region latency for clients far from home.

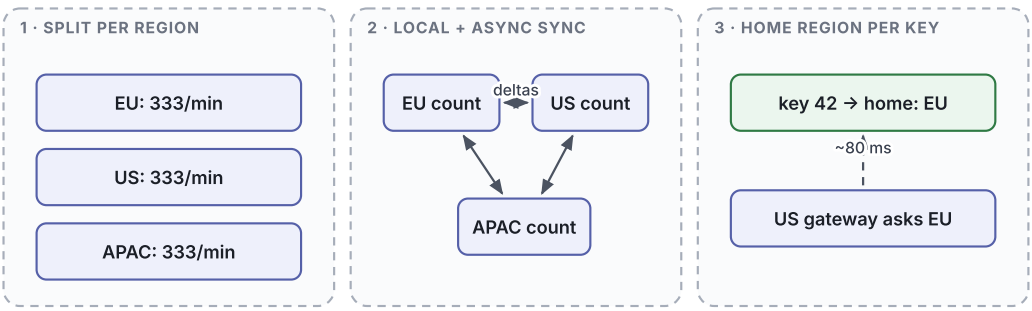


Figure 6.2 Three ways to enforce one limit across regions.

Most platforms use (1) or (2) for general limits, and keep strict security limits in a single home region or a strongly consistent store.

Failing open or closed

If Redis is unreachable, the limiter must decide (Vol. 1, Ch. 9). Default rules **fail open** with a conservative **local** token bucket per gateway (limit ÷ number of gateways), so the API keeps working with rough protection. **Strict** rules (login, OTP, payments) **fail closed**, since letting brute-force attempts through is worse than a brief error.

Hot keys

One huge customer at 100k requests/s concentrates load on one Redis shard. Spread the key into *N* sub-buckets (`rl:key#0…#7`), each allowed limit ÷ N, with the gateway picking one at random. Or give that customer a local-allowance scheme that synchronises in batches.

Telling clients what happened

Return `429 Too Many Requests` , `Retry-After` , and `RateLimit-Limit / RateLimit-Remaining / RateLimit-Reset` headers. Well-behaved clients can back off accurately, and support teams can explain the behaviour. Log denials (sampled) with rule IDs for dashboards and abuse detection.

6.6 Step 6: Failure modes

Failure	Effect	Mitigation
Redis shard down	Limits for its keys unavailable	Replicas with failover; fail-open local fallback for normal rules
Network partition between regions	Global counts diverge	Bounded overshoot; reconcile when healed
Hot key	One shard saturated	Sub-bucket sharding; local allowances
Clock skew between gateways	Refill miscalculated	Use Redis server time inside the script (`TIME`)
Bad rule pushed	Legitimate traffic rejected	Staged rollout of rules, dry-run "shadow" mode, quick rollback

CHECK YOUR UNDERSTANDING

1. Why must the token-bucket update run as a single atomic operation (such as a Lua script) in Redis?

   See answer →

2. Give two ways to enforce one limit across multiple regions, with their trade-offs.

   See answer →

3. Which rules should fail open and which fail closed when the limiter's store is unavailable?

   See answer →

4. One customer sends 100,000 requests per second under a single API key. What problem does this cause, and how do you fix it?

   See answer →

5. How can you reduce the latency the limiter adds to every request?

   See answer →

PART III

Real-Time Experiences

Systems that must react within milliseconds to what millions of people are doing — delivering alerts, merging simultaneous edits, ranking players and predicting the next keystroke.

CHAPTER 7

↩ Contents

Notification System

Design a platform service that other teams use to send notifications (push, SMS, email and in-app) for events like "your order shipped", "new message", "password changed" and weekly digests.

7.1 Step 1: Requirements

Functional

- An API for internal services: send to a user, through one or more channels, from a template with parameters.
- Delivery through third-party providers: Apple and Google push services, SMS gateways, email providers.
- User preferences: opt in or out per channel and category, quiet hours, frequency caps.
- Scheduled sends and bulk campaigns (millions of users).
- Delivery tracking: sent, delivered, opened, failed.

Non-functional

- **Prioritised:** a one-time password must arrive within seconds, even while a 50M-user marketing campaign is going out.
- **At-least-once delivery, but no visible duplicates.** Users hate getting the same alert twice.
- Survive provider outages and rate limits.
- Scale: 1B notifications/day, with peaks of 100k/s during campaigns.

7.2 Step 2: API

```
POST /v1/notifications
  Idempotency-Key: order-981-shipped
  {user_id, category: "order_updates", template: "order_shipped",
   params: {order_id: 981, eta: "Fri"}, channels: ["push","email"],
   priority: "high" | "normal" | "bulk", send_at?: "2026-10-16T09:00:00Z"}
-> 202 {notification_id}
```

The API **accepts and queues**. It never sends synchronously. Callers get a 202 in milliseconds.

7.3 Step 3: High-level design

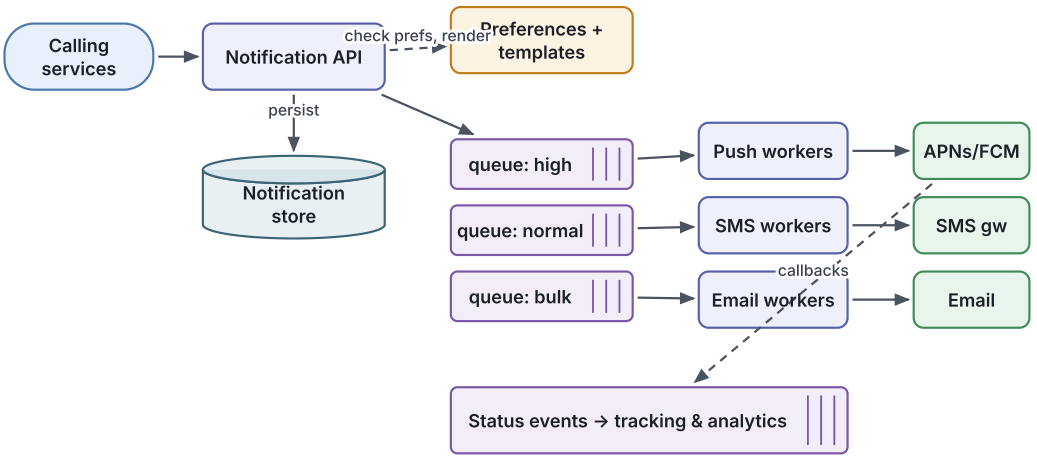


Figure 7.1 Notification pipeline. Requests are validated and stored, then flow through priority queues to channel workers that call external providers, with status events fed back.

1. The **API** validates the request, de-duplicates it by idempotency key, persists it, and works out which channels to use from the user's **preferences** (opted out? quiet hours? frequency cap reached?).
2. It renders the template for each channel (localised) and enqueues one **message per channel** onto a queue for its **priority**.
3. **Channel workers** pull from the queues and call the provider for that channel, through per-provider rate limiting and circuit breakers.
4. Provider responses and callbacks (delivered, bounced, device token invalid) flow back as **status events** to update tracking and clean up bad addresses.

7.4 Step 4: Deep dives

Priorities and isolation

Use **separate queues, and separate worker capacity, per priority class**. A 50M-message campaign fills the bulk queue but can't delay one-time passwords in the high queue, because high-priority workers never touch bulk work. This is the bulkhead pattern (Vol. 1, Ch. 18) applied to queues. Workers may take bulk work only when the higher queues are empty.

Fan-out for campaigns

"Send the weekly digest to 50M users" is one request that becomes 50M messages. A **campaign scheduler** pages through the target segment in batches (for example, 10k users), applies preferences, and enqueues messages at a controlled rate, spread over the send window and over time zones ("9 a.m. local time"). This protects the providers and our own downstream services, such as the deep-link pages users open.

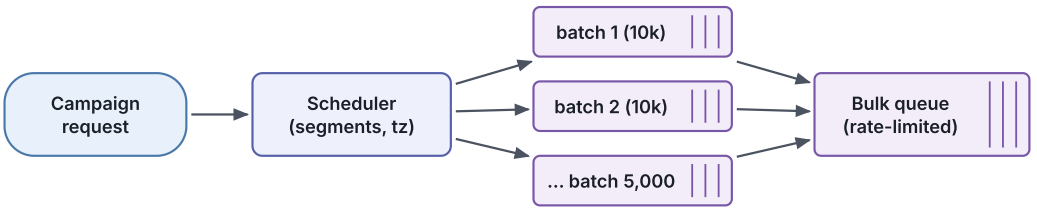


Figure 7.2 Campaign fan-out. One request becomes millions of messages, released in batches at a controlled rate and per time zone.

Avoiding duplicates

At-least-once queues mean a worker may send, crash before acknowledging, and the message gets redelivered. To stop users from seeing two copies:

- Record `(notification_id, channel)` as *sent* in a fast store **before acknowledging**, and check it before sending.
- Pass a **collapse key or provider-side de-duplication ID** where the provider supports one (push services can replace an earlier notification with the same collapse key).
- Make upstream callers idempotent with idempotency keys, so that a retried "order shipped" call doesn't create a second notification.

A tiny duplicate window can remain (the crash happens after the provider accepted the message but before we recorded it). Collapse keys narrow it further.

Provider failures and rate limits

Each provider limits our sending rate and sometimes fails. Workers use **token buckets per provider** to stay under limits, **exponential backoff with jitter** on transient errors, and **circuit breakers**. For critical messages, **fail over** to a secondary provider (a second SMS gateway). Permanent failures (invalid number, unregistered device token) are not retried. They update the user's contact record instead.

Preferences, quiet hours and caps

Preferences are read on every notification, so they must be cached (in Redis or locally with short TTLs). Quiet hours *delay* non-urgent notifications to the next allowed time (re-queued with a scheduled time) rather than dropping them. Frequency caps ("max 3 marketing pushes per day") are counters per user per category with a daily TTL. Transactional and security messages bypass marketing caps but still respect hard opt-outs where the law requires.

In-app notifications and the inbox

The in-app bell icon is a per-user **inbox**: a list sorted by time with read/unread state, stored in a wide-column store keyed by `user_id`. Online users get new items pushed over their existing WebSocket or SSE connection (Vol. 1, Ch. 10). Unread counts are cached counters.

7.5 Step 5: Failure modes

Failure	Effect	Mitigation
Campaign floods the system	OTPs delayed	Separate priority queues and worker pools
Provider outage	Channel down	Circuit breaker, retry with backoff, secondary provider for critical messages
Worker crash after send	Duplicate message	Sent-log check before send, collapse keys, idempotency
Stale device tokens	Wasted sends, provider penalties	Process provider feedback; remove invalid tokens
Preferences service slow	Pipeline stalls	Cache preferences; fail safe (skip marketing, still send security messages)

CHECK YOUR UNDERSTANDING

1. How do you guarantee that one-time passwords are delivered quickly while a huge marketing campaign is going out?

See answer →

2. Why does the notification API return 202 Accepted instead of sending synchronously?

   See answer →

3. Describe two techniques to stop users from receiving duplicate notifications under at-least-once delivery.

   See answer →

4. An SMS provider starts timing out on 30% of requests. How should the SMS workers react?

   See answer →

5. How should quiet hours be handled for a non-urgent notification?

   See answer →

CHAPTER 8

↩ Contents

Collaborative Document Editor

Design a real-time collaborative editor like Google Docs: several people edit the same document at once, see each other's changes and cursors within a fraction of a second, and nobody's work is lost.

8.1 Step 1: Requirements

Functional

- Create and open rich-text documents. Several users edit at the same time.
- See other users' edits live, plus their cursors and selections (presence).
- Offline editing that merges on reconnect.
- Version history: see and restore earlier versions. Comments and sharing (briefly).

Non-functional

- Low latency: your own typing must feel instant (local), and others' edits appear within ~100–300 ms.
- **Convergence:** all users eventually see exactly the same document.
- **Intention preservation:** merged edits should do what each user meant.
- Scale: 100M documents, 10M daily editors, usually 1–10 editors per document, occasionally hundreds.

8.2 Step 2: The core problem

Alice and Bob both see the text `cat`. At the same moment, Alice inserts "s" at the end (position 3) and Bob deletes the "c" at position 0. If each edit is simply applied on the other's machine *as sent*:

- Alice: `cats`, then Bob's "delete at 0" gives `ats` ✓
- Bob: `at`, then Alice's "insert s at 3" means position 3 doesn't exist in a 2-character string, so the result is wrong or an error ×

Concurrent edits refer to positions in different versions of the document. We need a way to merge them so that every replica converges. There are two families of solutions.

8.3 Step 3: Operational Transformation (OT)

With **OT**, when an operation arrives that was made concurrently with operations you've already applied, you **transform** it against them, adjusting its position so it still means the same thing. Bob's view: Alice's `insert("s", 3)` was concurrent with his `delete(0)`, which removed one character before position 3, so it is transformed to `insert("s", 2)`, giving `ats` ✓. Both sides converge.

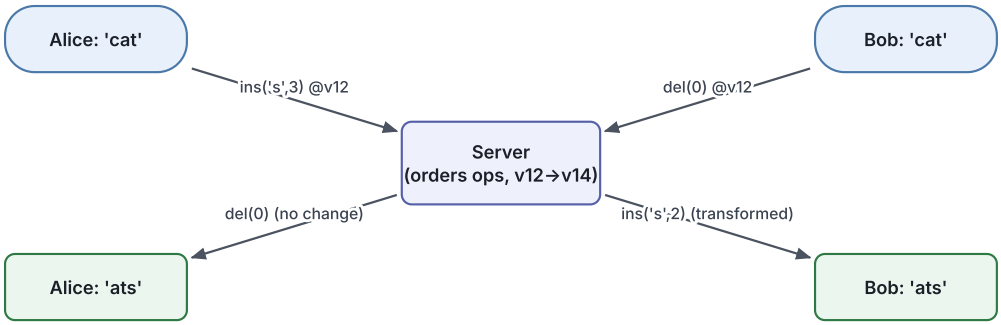


Figure 8.1 Operational transformation. A central server orders operations; concurrent operations are transformed against each other so both clients converge.

Practical OT systems use a **central server** that assigns every operation a position in one total order (a revision number). Clients send operations tagged with the revision they were based on. The server transforms each incoming operation against any operations committed since that revision, applies it, and broadcasts it. A central sequencer keeps the transformation rules manageable. OT without a central server is notoriously hard to get right.

8.4 Step 4: CRDTs

A **CRDT** (conflict-free replicated data type, Vol. 1, Ch. 14) designed for text gives every character a **unique, permanent identifier** (for example, `(site_id, counter)`) and defines positions *relative to identifiers* instead of indexes: "insert 's' after character `(alice, 3)`". Deletions mark characters as removed (tombstones) instead of shifting positions. Because operations refer to stable IDs and the merge rules are commutative, replicas can apply operations **in any order**, with no central server, and still converge.

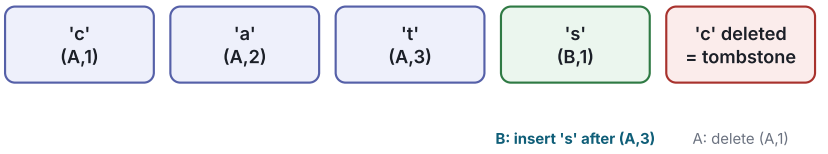


Figure 8.2 A text CRDT. Every character has a permanent ID, and inserts reference their neighbour's ID instead of a position.

	OT	CRDT
Positions	Indexes, transformed	Stable unique IDs
Server	Central sequencer (practically required)	Optional, peer-to-peer possible
Offline / P2P	Harder	Natural
Metadata overhead	Low	Higher (IDs, tombstones), mitigated by compaction
Maturity in big products	Long track record	Increasingly common in modern editors

TRADE-OFF

Which one?

A centralised, always-online editor can use OT with a server sequencer and keep documents compact. An editor that must work offline for long periods, sync device to device, or avoid a central bottleneck is a natural fit for CRDTs. Many modern systems use CRDTs and still route traffic through a server for persistence, access control and fan-out.

8.5 Step 5: High-level design

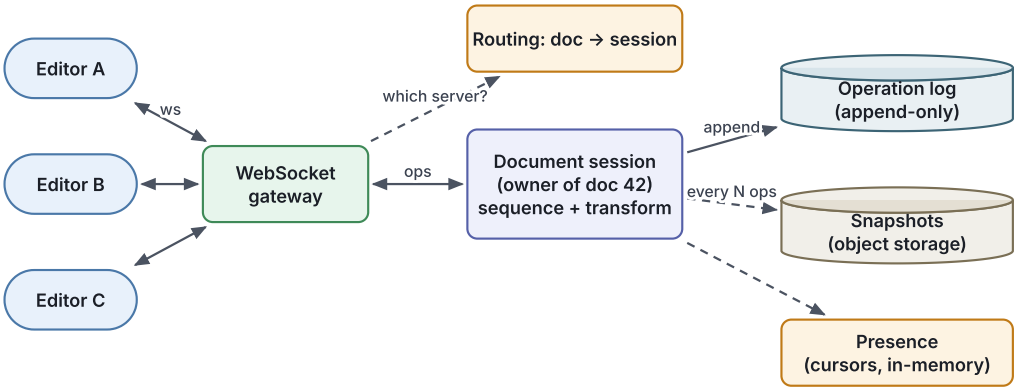


Figure 8.3 Collaborative editing architecture. Each open document is owned by one session server that sequences and broadcasts operations, persists them to an operation log, and periodically snapshots the document.

Document sessions: every open document is assigned to **one session server** (via consistent hashing on `doc_id`, or a routing table with leases). All editors of that document connect, through the gateway, to that server, which holds the live document in memory, orders operations, transforms or merges them, and broadcasts the results. One owner per document makes ordering simple and fast. Different documents spread across many servers.

Persistence: every accepted operation is appended to the document's **operation log** before it is acknowledged. Every few hundred operations, a **snapshot** of the full document is saved. Loading a document means reading the latest snapshot and replaying the operations after it. The operation log also gives **version history** for free.

Local-first typing: the client applies your keystrokes to its local copy immediately (so typing feels instant), sends them asynchronously, and reconciles with the server's ordered stream as acknowledgements and other users' operations arrive.

8.6 Step 6: Deep dives

Presence

Cursors and selections change constantly and don't need to be durable. They are sent as lightweight, throttled messages (for example, 10/s at most) through the session server and kept only in memory, with a TTL when users go idle. Losing one costs nothing, because the next update replaces it.

Offline editing

The client queues its operations locally while offline. On reconnect, it sends them tagged with its last known revision (OT), or simply exchanges missing operations (a CRDT state-vector sync). The server merges them. For long offline sessions, CRDTs make this merge straightforward.

Session server failure

If a session server dies, its documents are reassigned (the lease expires and the routing table points to a new owner). The new owner loads the snapshot plus the operation log. Clients reconnect and resend any operations that weren't acknowledged. Since accepted operations were persisted before being acknowledged, nothing confirmed is lost, and resent operations are de-duplicated by `(client_id, seq)`.

Very large audiences

A document with 500 simultaneous viewers mostly needs *fan-out*, not merging. Separate a small set of editors from many read-only viewers, and broadcast through a pub/sub tier or the CDN with a slight delay, so the session server isn't sending every keystroke to hundreds of sockets itself.

Access control

Permissions (owner, editor, commenter, viewer) are checked when a client joins a session, and again whenever they change, by kicking or downgrading connected clients. The session server rejects edit operations from users without edit rights.

8.7 Step 7: Failure modes

Failure	Effect	Mitigation
Concurrent conflicting edits	Divergence	OT transformation or CRDT merge rules guarantee convergence
Session server crash	Brief disconnect	Reassign doc; reload snapshot + log; clients resend unacked ops
Client offline for hours	Large merge	Local op queue; CRDT/state-vector sync
Operation log grows forever	Slow loads	Periodic snapshots; compact old history
Hundreds of viewers	Fan-out load	Separate viewers; pub/sub broadcast

CHECK YOUR UNDERSTANDING

1. Why can't concurrent edits simply be applied as sent on every replica?

   See answer →

2. How does a text CRDT avoid the position problem?

   See answer →

3. Why assign each open document to a single session server?

   See answer →

4. Why are cursor and selection updates handled differently from document edits?

   See answer →

5. How do snapshots and the operation log together support fast loading and version history?

   See answer →

CHAPTER 9

↩ Contents

Leaderboards and Live Metrics

Design the ranking and counting systems for a large online game: a global leaderboard of player scores updated in real time, rankings for friends and regions, and live dashboards of counts like "players online" and "unique players today".

9.1 Step 1: Requirements

Functional

- Submit a score after each match. A player's best score counts.
- Show the top 100 globally, a player's own rank ("you are #48,213"), the players just above and below them, and friends-only and regional boards.
- Daily, weekly and all-time leaderboards.
- Live metrics: concurrent players, matches per minute, unique daily players, top-K most played maps.

Non-functional

- Score updates are visible in under a second. Rank queries in under 50 ms.
- 50M players. 20k score updates/s at peak. 100k rank reads/s.
- Approximate answers are fine for metrics, and exact for the top of the leaderboard.

9.2 Step 2: The core structure: sorted sets

A **sorted set** keeps members ordered by score, with O(log n) insert, update and rank lookups. Redis implements it with a skip list plus a hash map (Vol. 1, Ch. 21).

```
ZADD   lb:alltime GT 9120 player:42        # set score, only if greater than
the current one
ZREVRANGE lb:alltime 0 99 WITHSCORES       # top 100
ZREVRANK  lb:alltime player:42             # 0-based rank of player 42
ZREVRANGE lb:alltime (rank-5) (rank+5)     # neighbours around the player
```

50M members × ~100 bytes ≈ 5 GB, which fits in the memory of one large node.

9.3 Step 3: High-level design

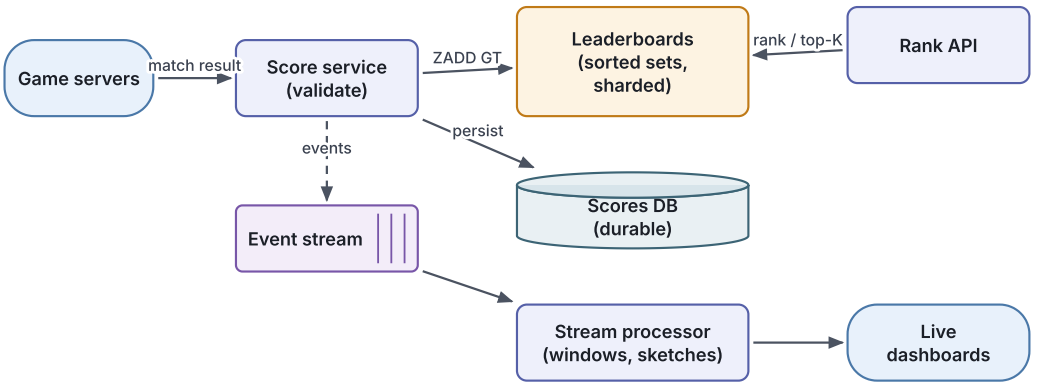


Figure 9.1 Leaderboard and metrics architecture. Score events update sorted sets in real time, and a stream processor computes approximate live metrics with sketches.

Game servers submit results to the **score service**, which validates them (anti-cheat: server-authoritative results, plausibility checks), persists them durably, updates the sorted sets, and emits an event. The **rank API** answers reads directly from the sorted sets. The **stream processor** consumes events to produce live metrics.

9.4 Step 4: Deep dives

Time-windowed boards

Keep a separate sorted set per period: `lb:daily:2026-10-15`, `lb:weekly:2026-W42`, `lb:alltime`. Each score update writes to all of the current ones (pipelined). Old period keys get a TTL and are archived to the database for history. Nothing needs to be "reset". A new day simply starts a new key.

Scaling beyond one node

When one sorted set gets too big or too hot:

- **Shard by score range:** shard 1 holds scores 0–999, shard 2 holds 1000–4999, and so on. A player's rank = their rank within their shard + the total number of members in all higher shards (kept in cached counters). Top-K reads come from the highest shard alone. Score changes may move a player between shards.

- **Shard by player and merge for top-K:** each shard holds a slice of players. The global top 100 = merge each shard's top 100. Exact rank for an arbitrary player requires asking every shard "how many scores are above X?" and adding up the answers. That is fine for a few shards, expensive for many.
- **Approximate rank for the long tail:** few people care whether they are #4,812,113 or #4,812,500. Keep exact sorted sets for the top tier, and estimate deep ranks from a score histogram ("top 9.6%").



Figure 9.2 Sharding a leaderboard by score range. Rank = rank within your shard + the members of all higher shards.

> TRADE-OFF
>
> **Exact where people look, approximate where they don't**
>
> Players scrutinise the top of the board and their friends' ranks, so those must be exact and fresh. Deep global ranks can come from a histogram or percentile estimate, which is far cheaper and good enough. Matching precision to attention is a recurring pattern in large systems.

Friends and regional boards

Regional: a separate sorted set per region, updated alongside the global one. **Friends:** most players have fewer than a few hundred friends, so compute on read: fetch each friend's score (a multi-get from a hash) and sort in memory, cached briefly per player. Avoid maintaining a sorted set per friendship graph.

Durability and rebuilds

The sorted sets are a fast **derived view**. The durable source of truth is the scores database, or the event log. If the cache cluster is lost, rebuild the sets by replaying best scores from the database. Redis persistence and replicas reduce how often this is needed.

Live metrics with sketches

- **Concurrent players:** each game server reports its connected count every few seconds, and the dashboard sums the latest reports (gauges, Vol. 1, Ch. 24).

- **Matches per minute:** tumbling one-minute windows in the stream processor (Vol. 1, Ch. 22).
- **Unique daily players:** a **HyperLogLog** per day (`PFADD dau:2026-10-15 player:42`) holds ~12 KB regardless of count, and daily sketches merge into weekly ones (Vol. 1, Ch. 21).
- **Top-K most played maps:** a **Count-Min Sketch** plus a small heap of candidates gives approximate heavy hitters in fixed memory.

9.5 Step 5: Failure modes

Failure	Effect	Mitigation
Cheated scores	Corrupted board	Server-authoritative results, validation, anomaly detection, moderation tools
Cache node lost	Board unavailable	Replicas + failover; rebuild from durable scores
Hot sorted set	Latency spikes	Read replicas for rank reads; shard by score range
Event backlog	Stale dashboards	Autoscale consumers; dashboards show data freshness

CHECK YOUR UNDERSTANDING

1. Why is a sorted set (skip list) a natural fit for a leaderboard?

   See answer →

2. How do you implement daily and weekly leaderboards without a reset job?

   See answer →

3. How can you compute an exact global rank when the leaderboard is sharded by score range?

   See answer →

4. Which structures would you use to count unique daily players and the most-played maps in fixed memory?

   See answer →

CHAPTER 10

↩ Contents

Search Autocomplete

Design the "type-ahead" suggestion box of a search engine or large e-commerce site: as the user types, show the 10 most relevant completions of their prefix within a few tens of milliseconds.

10.1 Step 1: Requirements

Functional

- Given a prefix, return the top 10 suggestions ranked by popularity, with some freshness (trending queries rise quickly) and personalisation.
- Support multiple languages, and filter offensive or unsafe suggestions.

Non-functional

- **Very low latency:** under 50 ms end to end, since a request fires on nearly every keystroke.
- High availability. Slightly stale suggestions are fine.
- Scale: 500M searches/day; with an average of ~6 keystroke requests per search, that is **3B requests/day ≈ 35k/s**, peaking around 100k/s.

10.2 Step 2: Core structure: a trie with cached top-K

A **trie** (prefix tree) stores strings character by character, so all queries sharing a prefix share a path. Walking to the node for "sys" finds every query starting with "sys". Collecting and sorting all of them for every keystroke would be too slow for short prefixes, which have millions of descendants. So **each node stores its own precomputed top-K list**.

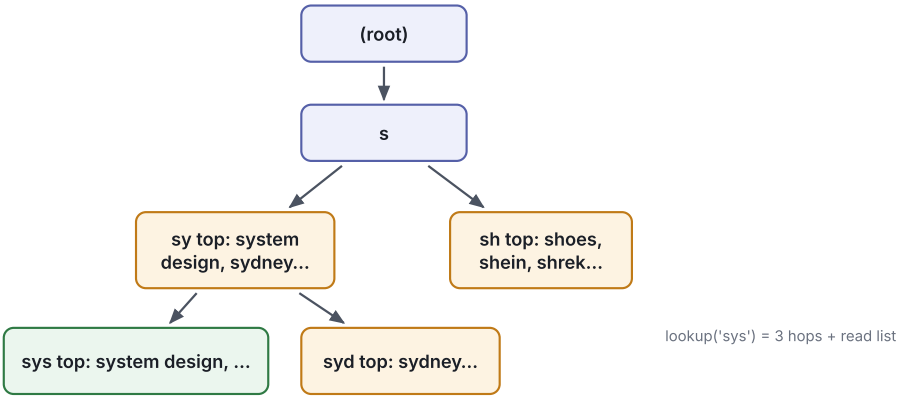


Figure 10.1 A trie where every node caches the top completions beneath it. A lookup is a walk down the prefix plus one read, with no subtree search.

A lookup is O(prefix length): walk three nodes and return the cached list. The cost moves to **building** the trie, which happens offline.

10.3 Step 3: High-level design

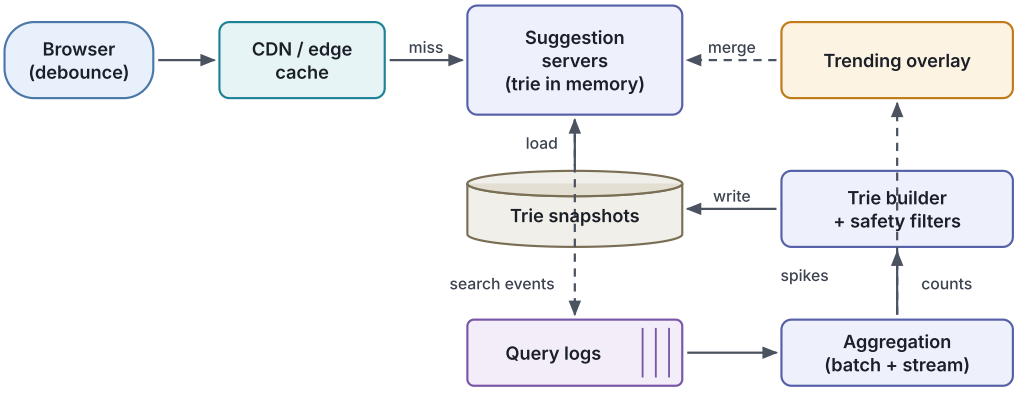


Figure 10.2 Autocomplete architecture. A data pipeline builds tries from query logs, suggestion servers hold them in memory, and caching at every layer absorbs repeated prefixes.

Serving path: the browser waits ~100 ms of typing inactivity (debounce) and caches responses locally, then asks the CDN edge, which caches popular prefixes for a few minutes. Misses reach the suggestion servers, which hold the trie in memory and answer in microseconds.

Data path: search events go to a log. Aggregation jobs count query frequencies over time windows. A builder produces a new trie (with top-K lists computed bottom-up), applies safety filters, and writes a versioned snapshot. Suggestion servers load the new version in the background and switch to it atomically.

10.4 Step 4: Deep dives

Ranking

Score each query by *frequency with time decay* (recent searches weigh more), plus signals such as click-through rate on the results. The top-K at each node is computed bottom-up: a node's list is the best K entries among its own query (if it is a complete query) and its children's lists.

Freshness: batch plus trending

A full rebuild from weeks of logs might run hourly or daily. Breaking news can't wait that long. A **streaming path** counts queries in short sliding windows (Vol. 1, Ch. 22), detects sudden spikes, and publishes a small **trending overlay** that suggestion servers merge with the base trie's results at query time. This is the Lambda-architecture idea in miniature: a slow, accurate batch layer and a fast, approximate speed layer.

Scaling the trie

- **Replicate:** the trie for one language and market is often a few GB, so full copies on many servers handle read throughput, with load balancing between them.
- **Shard when too big:** partition by prefix range ("a–f", "g–m", …), weighted by traffic rather than letters, since "s" is far busier than "x". A router sends each prefix to its shard.
- **Separate tries** per language or market (and per product vertical, where relevant).



Figure 10.3 Serving blends a slow, complete base trie with a fast trending overlay.

Personalisation

Blend the global suggestions with the user's own recent searches (kept client-side or in a small per-user store) at the edge or in the browser, so the shared cache stays effective and the trie stays global.

Safety

Suggestions are a product surface that can embarrass or harm. Filter offensive, hateful and personal-data queries with blocklists and classifiers during the build, apply an emergency blocklist at serving time for instant removals, and require a minimum query frequency from many distinct users, so that a few people can't game suggestions.

10.5 Step 5: Failure modes

Failure	Effect	Mitigation
Builder produces bad trie	Wrong or empty suggestions	Validate snapshots (size, sample queries) before rollout; keep previous version for instant rollback
Suggestion server overload	Slow typing experience	Debounce, client and edge caching, more replicas
Offensive suggestion surfaces	Reputational harm	Build-time filters, serving-time emergency blocklist
Trending pipeline lags	Stale for breaking events	Base trie still serves; monitor freshness

CHECK YOUR UNDERSTANDING

1. Why does each trie node store a precomputed top-K list?

   See answer →

2. How do you keep suggestions fresh for breaking news when the main trie is rebuilt only hourly?

   See answer →

3. List three layers of caching or request reduction in front of the suggestion servers.

   See answer →

4. How would you stop a small group of users from pushing an offensive phrase into the suggestions?

   See answer →

5. Why shard a trie by traffic-weighted prefix ranges rather than equal alphabet ranges?

See answer →

PART IV

Putting It Together

The handful of building blocks that keep reappearing across all ten designs, and how to recognise which ones a new problem needs.

CHAPTER 11

↩ Contents

Patterns Across Problems

After ten case studies, a surprising fact stands out: the systems look different on the surface, but they are built from the same dozen or so recurring combinations. Learning to spot these combinations is what turns a new, unfamiliar problem into a familiar one.

11.1 Pattern 1: Accept fast, process later

Shape: the API validates, persists and enqueues, and returns immediately (202 Accepted or PENDING). Workers do the slow work asynchronously.

Seen in: video transcoding, notifications, the crawler's parse step, payment webhooks.

Why it works: it keeps user-facing latency low, absorbs spikes in queues, and isolates users from slow or failing dependencies (Vol. 1, Ch. 11).

What to watch: queue depth and consumer lag; idempotent workers; dead-letter queues; telling the user about eventual completion (status pages, push notifications).

11.2 Pattern 2: Separate the control plane from the data plane

Shape: small, critical metadata goes through a transactional service. Huge payloads go directly between clients and blob storage or CDNs, authorised by tokens or pre-signed URLs.

Seen in: video uploads, file sync (metadata service vs. block service), CDN delivery of segments.

Why it works: each plane scales and fails independently. Application servers never carry gigabytes. Metadata keeps strong guarantees while blobs get cheap, massive throughput.

11.3 Pattern 3: Atomic conditional writes guard scarce resources

Shape: "change X from state A to state B *only if* it is still in state A", as one atomic operation (compare-and-set, `UPDATE … WHERE status = …`, a version check).

Seen in: seat holds, payment state transitions, file-version commits, driver assignment (Vol. 1, Ch. 29), token-bucket updates in Lua.

Why it works: exactly one competitor wins and the others fail fast, without long-held locks.

Pair it with: expiry (leases, holds) so that abandoned claims return on their own.

11.4 Pattern 4: Idempotency at every boundary

Shape: every request that changes something carries a unique key. The receiver records keys and returns the stored result for repeats.

Seen in: payments (client → service → PSP), notifications, file chunk uploads (the hash *is* the key), collaborative edit operations `(client_id, seq)`, crawler fetch tasks.

Why it works: networks force retries, retries create duplicates, and idempotency makes duplicates harmless (Vol. 1, Ch. 8 and 11).

11.5 Pattern 5: Durable log as the source of truth, with derived views

Shape: append every change to a durable, ordered log or journal. Build fast, disposable views (caches, sorted sets, search indexes, tries) from it, and rebuild them by replaying.

Seen in: the file-sync namespace journal, the collaborative-editing operation log with snapshots, leaderboards rebuilt from durable scores, autocomplete tries rebuilt from query logs, the payment ledger.

Why it works: the log gives ordering, auditability and recovery. Views can be optimised freely for reads, because they can always be regenerated (Vol. 1, Ch. 17).

11.6 Pattern 6: The outbox for reliable side effects

Shape: write the state change and its outgoing event in one local transaction, and relay events afterwards (Vol. 1, Ch. 20).

Seen in: payment success → fulfilment, inventory changes → seat-map cache, notification status updates.

Why it works: it removes the dual-write gap where state and events disagree after a crash.

11.7 Pattern 7: Precompute for the read path

Shape: do expensive work at write time or offline so that reads become lookups.

Seen in: trie nodes caching top-K, transcoded renditions, news-feed fan-out (Vol. 1, Ch. 27), seat-map read models, daily leaderboard keys.

Why it works: most systems read far more than they write. Moving work off the read path buys latency and capacity where it matters.

Watch: write amplification and staleness. Use hybrid approaches (precompute for the common case, compute on read for the extremes).

11.8 Pattern 8: Exact where it matters, approximate everywhere else

Shape: spend precision only on what users scrutinise or what correctness requires. Use sketches, sampling and estimates for the rest.

Seen in: exact top-of-leaderboard vs. estimated deep ranks; HyperLogLog unique players; Bloom-filter seen-URLs; asynchronous view counts; approximate global rate limits with strict login limits.

Why it works: approximate structures cost orders of magnitude less memory and coordination (Vol. 1, Ch. 21).

11.9 Pattern 9: Meter the flood

Shape: when demand can exceed capacity by orders of magnitude, control admission instead of trying to absorb it: waiting rooms, rate limits, priority queues, campaign schedulers, debouncing.

Seen in: ticket on-sales, notification campaigns, rate limiting, autocomplete debouncing, crawler politeness.

Why it works: systems degrade gracefully and fairly instead of collapsing for everyone.

11.10 Pattern 10: One owner per hot object

Shape: route all activity for one entity (a document, a host, an event's inventory, a chat) to a single owner process, chosen by consistent hashing or a lease-based routing table.

Seen in: collaborative document sessions, crawler hosts, ticket inventory sections, chat conversations (Vol. 1, Ch. 28).

Why it works: a single owner can order operations and enforce invariants locally, without distributed coordination. Different entities spread across many owners, so the system still scales out.

Watch: failover (leases, fencing), and entities too hot for one owner (split them).

11.11 Pattern 11: Reconcile, don't just trust

Shape: periodically compare your state against an independent source of truth and repair the differences.

Seen in: payment reconciliation against PSP settlement files, the crawler re-checking freshness, anti-entropy between replicas (Merkle trees, Vol. 1, Ch. 21), clients polling the journal in case a notification was missed.

Why it works: every real system drifts. Request-path guarantees reduce drift, and reconciliation catches what slips through.

11.12 A recognition table

If the problem has…	Reach for…
Big files or media	Pattern 2 (pre-signed uploads, object storage, CDN), Pattern 1 (async processing)
Money, inventory, uniqueness	Patterns 3, 4, 6, 11
Many users editing shared state	Pattern 10 (owner per object), plus OT/CRDT, Pattern 5 (operation log)
Read-heavy, latency-critical lookups	Pattern 7 (precompute), caching, CDN
Huge counts or sets	Pattern 8 (sketches, approximate)
Sudden extreme spikes	Pattern 9 (metering, queues, waiting rooms)
Slow external dependencies	Pattern 1, circuit breakers, retries with idempotency
Anything that must be auditable	Pattern 5 (append-only log), double-entry where money is involved

INTERVIEW TIP

The meta-skill

Faced with a new problem, ask four questions: *What is scarce?* (Correctness patterns.) *What is huge?* (Data-plane and approximation patterns.) *What is spiky?* (Metering and async patterns.) *What is shared between users?* (Ownership and merge patterns.) The answers usually name most of the architecture before you draw a single box.

CHECK YOUR UNDERSTANDING

1. Name three case studies that rely on "atomic conditional writes", and say what resource each one protects.

   See answer →

2. What do the file-sync journal, the collaborative-editing operation log and the payment ledger have in common?

   See answer →

3. Give an example of "exact where it matters, approximate elsewhere" and explain why it is acceptable.

   See answer →

4. Why is "one owner per hot object" scalable even though each object has a single owner?

See answer →

APPENDIX

↩ Contents

Practice Problem Bank

Ten more design problems for practice. For each, the hints point at what makes the problem interesting and which patterns from Chapter 11 and chapters of both volumes apply. Try each one for 40 minutes before reading the hints.

1. Ride-sharing surge pricing service

Compute a price multiplier for every small area of a city every minute, from live demand and supply. *Hints:* geospatial cells (H3); streaming windows over request and driver events; smoothing and caps; serving multipliers from a cache; what happens if the stream lags (use the last known value). Vol. 1, Ch. 21–22, 29.

2. Distributed job scheduler (cron at scale)

Run millions of scheduled jobs ("every day at 9:00 in the user's time zone") exactly once each, on a fleet of workers. *Hints:* time-bucketed job tables or a timing wheel; leader election or partition ownership for scanning; leases and fencing so that a job isn't run twice; idempotent job handlers; catching up after downtime. Vol. 1, Ch. 19; Patterns 3, 4, 10.

3. Hotel reservation system

Book rooms by room type and date range, allowing controlled overbooking. *Hints:* inventory per (hotel, room type, date) counters rather than individual rooms; atomic decrement across all nights of a stay (one transaction, or a saga); holds with expiry; idempotent reservation requests. Chapter 5; Patterns 3, 4.

4. Metrics monitoring and alerting system

Ingest millions of metric points per second from servers, store them, graph them and alert on thresholds. *Hints:* time-series storage (compression, downsampling, retention tiers); label cardinality limits; a pull vs. push collection model; alert evaluation as a stream; alerts

deduplicated and routed through a notification system. Vol. 1, Ch. 24; Chapter 7.

5. Typeahead for people search inside a company

Like autocomplete, but over 500k employee names with permissions (some people are hidden from some viewers). *Hints:* a small dataset (it fits in memory everywhere); filtering by permission at query time; prefix matching on first and last names and nicknames; personal ranking (your team first). Chapter 10.

6. Online auction (eBay-style)

Users bid on items, and the highest bid at closing time wins. *Hints:* a single owner per auction for ordering bids (Pattern 10); atomic "accept a bid only if higher than the current one"; anti-sniping extensions; fan-out of live price updates over WebSockets; a precise close time with a server-side clock. Chapters 5, 8.

7. URL-shortener analytics at 1M clicks/s

Real-time click counts by link, country and referrer, with daily unique visitors. *Hints:* log ingestion; streaming aggregation into OLAP storage; HyperLogLog per link per day; Count-Min Sketch for top referrers; pre-aggregated rollups. Vol. 1, Ch. 26; Chapter 9.

8. Code deployment system

Push new builds to 10,000 servers safely within 10 minutes. *Hints:* artifact distribution (peer-to-peer or tiered caches, like a CDN); canary waves with automatic rollback on health metrics; idempotent deploy agents that converge to a desired version; a deploy-lock per service. Vol. 1, Ch. 23.

9. Multiplayer game server matchmaking

Group players of similar skill and region into matches of 10 within 30 seconds. *Hints:* queues per region and skill bucket; widening the skill window over time; atomically claiming players so they aren't in two matches (Pattern 3); allocating game servers; handling players who drop during matching.

10. Email service (Gmail-style)

Receive, store, search and send email for 1B users. *Hints:* SMTP ingestion; spam filtering pipeline (async); mailbox storage sharded by user, with blobs in object storage; inverted index per user for search; threads and labels as metadata; push to clients. Chapters 2, 7; Vol. 1, Ch. 14–16.

APPENDIX

↩ Contents

Answer Key

Every answer links back to its question. Try each question honestly before reading the answer — retrieval is what makes it stick.

Chapter 1: Video Streaming Platform

1. Videos are gigabytes in size. Streaming them through application servers would waste huge amounts of their bandwidth, memory and CPU and limit throughput. Pre-signed URLs let clients upload straight to object storage, which is built for large transfers, while the application still controls who may upload what. Multipart uploads also make large uploads parallel and resumable.

   ↩ Back to question

2. The video is split at keyframes into many short chunks, and each chunk-rendition pair is encoded as an independent task on a large worker pool at the same time. Total time is roughly the time to encode one chunk plus splitting and packaging, rather than the time to encode the whole video serially. Failed tasks are retried individually.

   ↩ Back to question

3. They are small, immutable files with stable, unique URLs. Their content never changes once published, so they can be cached for a very long time with no invalidation, and the same segment serves every viewer watching that rendition.

   ↩ Back to question

4. The event rate is enormous (tens of thousands per second, with very hot keys for viral videos), and exact real-time counts aren't needed. Logging events and aggregating them in a stream processor allows de-duplication and bot filtering, batches writes, avoids hot-row contention in the database, and keeps playback independent of the counting system. The cost is a count that lags slightly.

   ↩ Back to question

Chapter 2: File Storage and Sync

1. (1) Delta sync: only changed chunks are uploaded after an edit. (2) Deduplication: identical chunks, within or across files and users, are stored once. (3) Resumability and parallelism: transfers proceed chunk by chunk, so interruptions resume where they stopped and chunks can be transferred in parallel. Chunk hashes also verify integrity.

   ↩ Back to question

2. The journal gives a total order of every change in a namespace. Each device remembers the last sequence number it has applied and pulls everything after it, which makes sync incremental, resumable and correct even after long offline periods or missed notifications. It also makes notifications simple: they only need to say "something changed".

↩ Back to question

3. The first to commit succeeds. The second commit fails the base-version check, so the client saves its edit as a "conflicted copy" next to the synced version. Arbitrary binary formats can't be merged safely and automatically, and keeping both guarantees nothing is lost. The user resolves the conflict.

↩ Back to question

4. With fixed-size chunks, any insertion near the start shifts every later chunk boundary, so every chunk hash changes and the whole file is re-uploaded. Content-defined chunking places boundaries based on the data itself, so only the chunks around the change get new hashes and everything after it deduplicates against the previous version.

↩ Back to question

5. Most operations (listing a folder, moving files within a tree, committing changes and appending to the journal, syncing) touch many files within one namespace and need transactions across them. Sharding by namespace keeps those operations on one shard, so they stay atomic and fast.

↩ Back to question

Chapter 3: Web Crawler

1. The two goals pull in different directions. Priority wants the most important URLs first, while politeness wants each host fetched slowly with a delay between requests. Front queues order URLs by priority. Back queues group URLs by host, and a heap of next-allowed times ensures each host is fetched at most at its polite rate, so fetchers stay busy across many hosts without overloading any one of them.

↩ Back to question

2. It stores tens of billions of URL hashes in a small fraction of the memory of an exact set, and answers in constant time, which is essential at thousands of links per second. A false positive means a genuinely new URL is wrongly treated as seen and skipped. For a crawler, missing a tiny fraction of pages is an acceptable trade for the huge savings.

↩ Back to question

3. Use trap defences: limit the crawl depth and number of pages per host, cap URL length, detect repeating URL patterns or parameters, and lower the priority of URLs whose content is near-duplicate or low value. Each host gets a budget, so runaway URL spaces can't swamp the frontier.

↩ Back to question

4. Exact copies are caught by a content hash. Near-duplicates are caught with similarity fingerprints such as SimHash or MinHash, where similar documents produce fingerprints that differ in only a few bits. Pages whose fingerprints are close are treated as duplicates: stored once, and their links not re-followed.

   ↩ Back to question

5. So that all URLs for a host live on one node, which can then enforce that host's politeness (crawl delay, concurrency limit) locally without coordinating with other nodes. Consistent hashing on the hostname spreads hosts evenly and limits reshuffling when nodes are added or removed.

   ↩ Back to question

Chapter 4: Payment System

1. Floating-point numbers can't represent most decimal fractions exactly ($0.1 + 0.2 \neq 0.3$), so repeated arithmetic accumulates rounding errors and amounts fail to balance. Integer cents (with an explicit currency) are exact, which accounting requires.

   ↩ Back to question

2. Every transaction is recorded as two or more entries (debits and credits) across accounts that sum to zero, so money only moves and is never created or destroyed. Any bug that loses or duplicates money produces an imbalance that a simple sum detects, and the append-only entries give a full audit trail.

   ↩ Back to question

3. The PSP may have charged the card even though the response was lost. Marking the payment failed would invite the customer to pay again, which is a double charge. Instead, keep the payment PENDING or UNKNOWN, then resolve it: query the PSP using the idempotency key, wait for the webhook, retry safely with the same key, and rely on reconciliation as a backstop.

   ↩ Back to question

4. Verify each signature, then de-duplicate by the webhook's event ID, so the second success is ignored. Apply transitions only through the state machine's guarded conditional updates: PENDING → SUCCEEDED is allowed, but SUCCEEDED → FAILED is not, so the late "failed" event is recorded for audit and rejected as a transition (and investigated if it indicates a real discrepancy).

   ↩ Back to question

5. Silent drift: missed or never-delivered webhooks, bugs that wrote wrong states or amounts, and PSP-side errors. Matching the PSP's settlement report line by line against the internal ledger finds discrepancies that no single request-path mechanism would notice.

   ↩ Back to question

Chapter 5: Ticket Booking System

1. Every change of a seat's state is an atomic conditional update in a strongly consistent store: AVAILABLE → HELD only `WHERE status='AVAILABLE'` (or the hold has expired), and HELD → SOLD only `WHERE hold_id` matches and the hold hasn't expired. Only one transaction can satisfy the condition for a given seat, so concurrent attempts lose and must choose again.

   ↩ Back to question

2. Users abandon checkouts, close tabs or lose connection. Holds that expire automatically return seats to sale without depending on the user or any cleanup message, like a lease. Indefinite locks would strand inventory and need fragile manual release.

   ↩ Back to question

3. (1) Overload: it meters users into the booking system at a rate the system can handle, instead of letting millions of simultaneous requests crush the database. (2) Fairness: positions are assigned by arrival or randomised among early arrivals, so raw speed and refresh-spamming give no advantage. (3) It gives a stable, informative experience (a queue position) served cheaply from the CDN.

   ↩ Back to question

4. The map is only for browsing. Correctness is enforced at hold time by the atomic conditional update, which rejects seats that were taken in the meantime. Serving the map from a cache keeps millions of reads off the inventory database, at the cost of occasional "seat already taken" responses.

   ↩ Back to question

Chapter 6: Distributed Rate Limiter

1. The update is a read-modify-write: read tokens, refill, compare and decrement. If two gateways do this at the same time as separate commands, both can read the same token count and both spend the last token, letting extra requests through. Running it atomically on the server serialises updates per key.

   ↩ Back to question

2. (1) Split the limit per region (limit ÷ regions): fast and simple, with no cross-region traffic, but a client using one region can't reach its full limit. (2) Count locally and sync deltas asynchronously between regions (CRDT counters): close to a true global limit, at the cost of temporary overshoot bounded by the sync interval. A third option, a single home region per key, is exact but adds cross-region latency.

   ↩ Back to question

3. General API usage limits should fail open (allow, ideally with a conservative local limit) so that a limiter outage doesn't become an API outage. Security-critical rules such as login attempts, one-time-password checks or payment attempts should fail closed, because allowing unlimited attempts would enable brute-force attacks.

↩ Back to question

4. All of that key's limiter operations land on one Redis shard, which becomes a hot spot and adds latency for every key on that shard. Split the key into N sub-buckets (`key#0` … `key#N-1`), each holding limit ÷ N, with gateways picking one at random; or give that customer local allowances on each gateway that sync in batches.

↩ Back to question

5. Run the limiter library inside the gateway with rules cached locally; use one pipelined round trip to a nearby (same-region) Redis; cache recent denials locally; and allow requests from keys far below their limit locally, reporting usage asynchronously in batches, so that only keys near their limit need a synchronous check.

↩ Back to question

Chapter 7: Notification System

1. Separate the traffic by priority: different queues and dedicated worker capacity (and provider quota) for high-priority transactional messages, so bulk campaign messages can never fill the queue ahead of them or use up their workers. Campaigns are also throttled by a scheduler. This is the bulkhead pattern.

↩ Back to question

2. Sending depends on slow, rate-limited, sometimes failing third-party providers. Accepting and queueing decouples callers from that: they get a fast response, spikes are absorbed by queues, retries and failover happen in the background, and calling services don't fail when a provider has an outage.

↩ Back to question

3. (1) Before sending, check and then record a sent marker for `(notification_id, channel)` in a fast store, so a redelivered message is skipped. (2) Use provider de-duplication features such as push collapse keys, so a repeat replaces the earlier notification instead of showing twice. Upstream idempotency keys also prevent callers from creating duplicate notifications.

↩ Back to question

4. Retry transient failures with exponential backoff and jitter, within per-provider rate limits. A circuit breaker opens when the failure rate crosses a threshold, so workers stop hammering the provider. Critical messages such as one-time passwords fail over to a secondary SMS provider, while bulk messages can wait in the queue until the breaker closes again.

↩ Back to question

5. Don't drop it: reschedule it for the end of the user's quiet window in their local time zone, by re-queueing it with a delayed send time. Urgent or security notifications may bypass quiet hours according to policy.

↩ Back to question

Chapter 8: Collaborative Document Editor

1. Operations refer to positions (indexes) in the document version their author saw. A concurrent edit by someone else may have inserted or deleted text before that position, so applying the operation unchanged puts it in the wrong place or makes it invalid, and the replicas diverge. Edits must be transformed (OT) or expressed relative to stable identifiers (CRDT).

↩ Back to question

2. Every character gets a globally unique, permanent ID, and insertions are expressed relative to neighbouring IDs ("after character X") rather than numeric indexes. Deletions mark characters as tombstones instead of shifting positions. Because the merge rules are commutative and idempotent, replicas can apply operations in any order and still converge, without a central sequencer.

↩ Back to question

3. One owner can impose a single total order on that document's operations, which keeps transformation and merging simple and fast, and it holds the live state in memory for low latency. Different documents spread across many servers, so the system scales horizontally. A failed owner is replaced by reassigning the document and reloading its snapshot and operation log.

↩ Back to question

4. Presence data changes many times per second, matters only while the user is active, and each update replaces the previous one. So it is throttled, kept only in memory with a TTL, and never written to the operation log. Losing an update costs nothing. Document edits, by contrast, must be durable, ordered and merged.

↩ Back to question

5. Every operation is appended to a durable log, which is a complete history that allows viewing or restoring any past version. Periodic snapshots save the full document state, so opening a document only needs the latest snapshot plus the operations after it, instead of replaying the entire history.

↩ Back to question

Chapter 9: Leaderboards and Live Metrics

1. It keeps members ordered by score and supports insert/update, rank lookup and range queries (top-K, neighbours around a player) in O(log n). That covers every leaderboard operation in real time for tens of millions of players held in memory.

↩ Back to question

2. Use a separate sorted-set key per period (such as `lb:daily:2026-10-15`) and write each score to the current period keys along with the all-time key. A new period simply starts writing a new key. Old keys expire through a TTL after being archived, so no reset is needed.

↩ Back to question

3. A player's rank equals their rank within their own shard plus the total number of members in all shards covering higher score ranges. Each shard's member count is kept in cached counters, so this needs only a lookup in one shard plus a sum.

↩ Back to question

4. A HyperLogLog per day for unique players (~12 KB, ~1% error, mergeable across days), and a Count-Min Sketch with a small heap of top candidates for approximate top-K map plays.

↩ Back to question

Chapter 10: Search Autocomplete

1. Short prefixes have millions of descendants, and collecting and sorting them on every keystroke would be far too slow. Caching the best K completions at each node makes a lookup a walk of length equal to the prefix plus one read, giving microsecond responses. The cost moves to the offline build.

↩ Back to question

2. Add a streaming path that counts recent queries over short sliding windows, detects spikes, and publishes a small trending overlay. Suggestion servers merge it with the base trie's results at query time. Batch gives accuracy, streaming gives speed.

↩ Back to question

3. (1) Debouncing in the browser, so not every keystroke fires a request. (2) A client-side cache of recent prefix responses. (3) CDN or edge caching of popular prefixes for a few minutes. Within the servers, the in-memory trie is itself a precomputed cache.

↩ Back to question

4. Require a minimum query frequency from many distinct users (and accounts or IPs) before a query can become a suggestion, filter candidates with blocklists and classifiers during the build, and keep an emergency serving-time blocklist so that anything harmful that gets through can be removed instantly without waiting for a rebuild.

↩ Back to question

5. Query volume is very uneven across starting letters: some prefixes get far more traffic than others. Equal alphabet ranges would overload some shards and leave others idle. Weighting shard boundaries by observed traffic balances the load.

↩ Back to question

Chapter 11: Patterns Across Problems

1. Ticket booking (seats: AVAILABLE → HELD only if still available); payments (payment state transitions such as PENDING → SUCCEEDED only from a valid prior state); file sync (committing a new version only if the base version is current). Others: the rate limiter's atomic token update, ride-hailing driver assignment.

   ↩ Back to question

2. Each is an append-only, ordered, durable log that serves as the source of truth. Current state or views are derived from it, it gives a full history and audit trail, and it allows recovery or catch-up by replaying from a known position.

   ↩ Back to question

3. Leaderboards: the top of the board and friends' ranks are exact, while deep global ranks are estimated from a score histogram. Users scrutinise the top and their friends, but nobody cares whether they are rank 4,812,113 or 4,812,500, so the cheap estimate loses nothing they value. Others: HyperLogLog unique counts, Bloom filters for seen URLs.

   ↩ Back to question

4. Ownership is per object, not global. Millions of documents, hosts or events spread across many owner processes through consistent hashing, so total capacity grows with the number of owners. Each owner orders and validates its own objects' operations locally with no cross-node coordination, and only exceptionally hot single objects need further splitting.

   ↩ Back to question